

CRUD GETX DENGAN FIREBASE

1. Pengenalan Firebase

Pada saat ini banyak bermunculan database dengan basis NoSQL (bukan sql), salah satunya adalah Firebase. Beberapa tahun terakhir ini Firebase mulai banyak dikenal dan digunakan oleh para developer termasuk di Indonesia.

Perkembangan teknologi tentunya memudahkan developer seperti dengan adanya Database Firebase ini. Namun, yang paling penting dari sebuah perkembangan adalah proses.

a. Mengenal Firebase



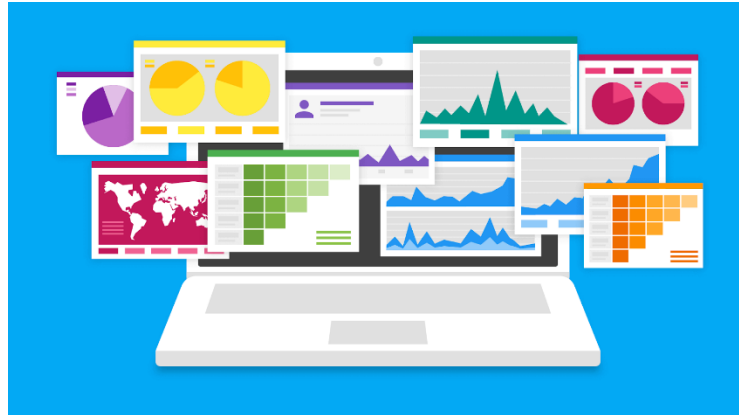
Firebase adalah suatu layanan dari Google untuk memberikan kemudahan bahkan mempermudah para developer aplikasi dalam mengembangkan aplikasinya. Firebase alias BaaS (Backend as a Service) merupakan solusi yang ditawarkan oleh Google untuk mempercepat pekerjaan developer.

Dengan menggunakan Firebase, apps developer bisa fokus dalam mengembangkan aplikasi tanpa memberikan effort yang besar untuk urusan backend.

Singkat cerita mengenai sejarah dari Firebase didirikan pertama kali pada tahun 2011 oleh Andrew Lee dan James Tamplin. Produk Firebase yang pertama kali adalah Realtime Database. Realtime Database digunakan developer untuk menyimpan data dan synchronize ke banyak user. Kemudian ia berkembang sebagai layanan pengembang aplikasi. Pada bulan Oktober 2014, perusahaan tersebut diakuisisi oleh Google.

Mengenai segi layanan, dulu Firebase memberikan service trial (percobaan), namun saat ini kamu bisa memanfaatkan dan menggunakan layanan Firebase secara free (gratis). Tentu saja dengan adanya batasan-batasan tertentu.

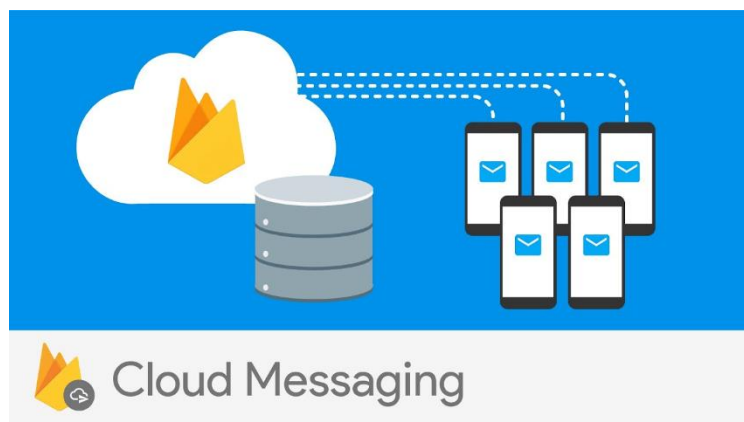
b. Firebase Analytics



Fitur Analytics adalah salah satu fitur pada Firebase yang digunakan sebagai koleksi data dan reporting untuk aplikasi Android maupun iOS. Koleksi data pun bervariasi. Sebagai contoh, kamu dapat membuat suatu laporan atau report untuk pengguna aplikasi di negara Indonesia saja, atau mungkin negara lain seperti Singapura. Kamu juga bisa melihat bagian mana saja dari aplikasi yang paling sering digunakan oleh user.

Fitur ini mempunyai kelebihan yang memungkinkan kita untuk bisa membuat segmentasi user berdasarkan user attribute. User attribute adalah suatu parameter yang bisa kita gunakan sebagai filter yang bertujuan untuk reporting dan notifikasi. Contohnya pada aplikasi online shop. Dengan user attribute, kamu bisa tahu jumlah user yang membeli handphone merk 'O' atau bahkan bisa mencari tahu jam berapa transaksi yang dilakukan user sering terjadi.

c. Firebase Cloud Messaging and Notifications



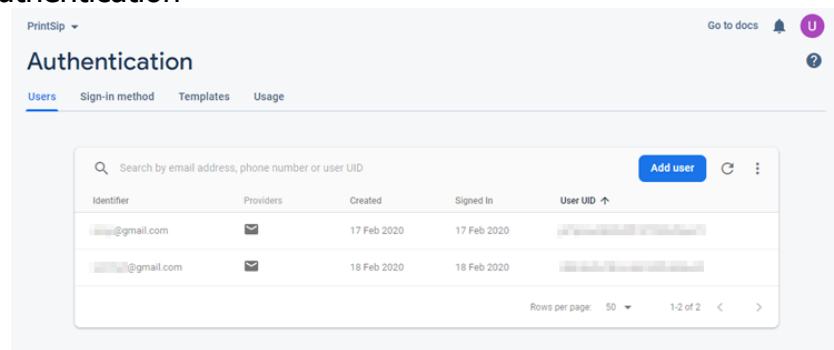
FCM (Firebase Cloud Messaging) yaitu menyediakan koneksi yang handal dan tentunya hemat baterai antar server maupun antar device. Sehingga kamu

dapat mengirim dan menerima pesan serta notifikasi di Android, iOS, dan web tanpa perlu biaya.

Untuk menargetkan pesan lanjutan, kamu bisa targetkan pesan dengan mudah menggunakan segment yang telah ditentukan sebelumnya yakni menggunakan demografi dan behavior/perilaku. Anda dapat menargetkan pesan ke perangkat yang telah berlangganan pada topik tertentu. Selain itu, Anda bisa juga menargetkan hanya ke satu perangkat untuk mendapatkan informasi data yang terperinci. Biasanya ini dilakukan untuk proses pengujian.

Pesan notifikasi ini terintegrasi sepenuhnya dengan Google Analytics for Firebase, sehingga kamu memiliki akses pada interaksi dan tracking konversi secara detail. Nah, Anda dapat memantau suatu efektivitas dari satu dashboard tanpa perlu coding atau membuat program sendiri.

d. Firebase Authentication



Firebase Authentication adalah salah satu layanan back-end, fitur Android dan iOS, SDK yang mudah digunakan, dan tampilan interfaces yang siap pakai untuk mengautentikasi pengguna ke aplikasi yang kamu buat. Firebase Authentication mendukung autentikasi menggunakan nomor telepon, sandi, penyedia identitas gabungan populer seperti seperti Google, Facebook, dan sebagainya.

Firebase Authentication terintegrasi dengan fitur layanan Firebase lainnya. Sistem ini memanfaatkan berbagai jenis standar industri, seperti OAuth 2.0 dan OpenID Connect, yang memudahkan integrasi dengan backend khusus buatanmu.

Kamu juga dapat memudahkan pengguna untuk login ke aplikasi dengan menggunakan fitur Firebase UI (tampilan interfaces), sebagai alternatif full drop-in authentication.

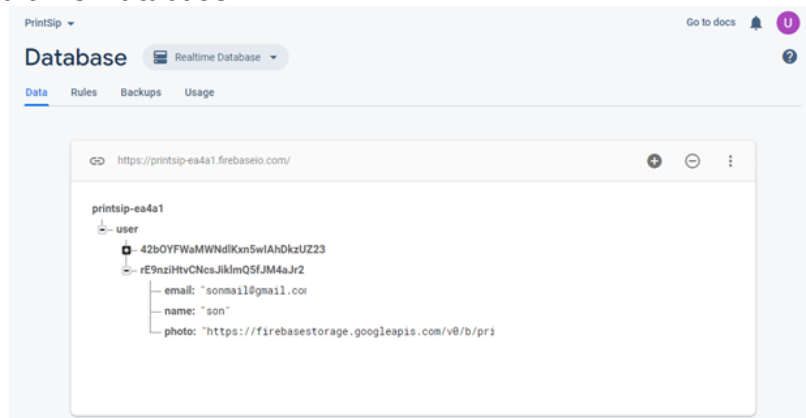
e. Firebase Cloud Firestore



Cloud Firestore adalah database yang bersifat fleksibel dan terukur untuk pengembangan perangkat seperti seluler, web, dan server di Firebase dan Google Cloud Platform. Seperti halnya Firebase Realtime Database, Cloud Firestore membuat datamu tetap terkoneksi di aplikasi user melalui listener realtime dan menawarkan layanan secara offline untuk aplikasi seluler dan web. Dengan begitu, kamu dapat membuat aplikasi yang powerful, responsif, dan mampu bekerja tanpa bergantung pada latensi koneksi internet.

Cloud Firestore merupakan database NoSQL yang dihosting di cloud dan dapat diakses melalui SDK real oleh aplikasi iOS, Android dan web.

f. Firebase Realtime Database



Firebase Realtime Database adalah database yang di-host melalui cloud. Data disimpan dan dieksekusi dalam bentuk JSON dan disinkronkan secara realtime ke setiap user yang terkoneksi. Hal ini berfungsi memudahkan kamu dalam mengelola suatu database dengan skala yang cukup besar. Ketika kamu membuat aplikasi lintas-platform/multiplatform menggunakan SDK Android, iOS, dan juga JS (JavaScript), semua pengguna akan berbagi sebuah instance Realtime Database dan menerima update-an data secara serentak dan otomatis.

Kemampuan lain dari Firebase Realtime Database adalah tetap responsif bahkan saat offline karena SDK Firebase Realtime Database menyimpan data langsung ke disk device atau memori lokal. Setelah perangkat terhubung kembali dengan internet, perangkat pengguna (user) akan menerima setiap perubahan yang terjadi.

g. Firebase Hosting



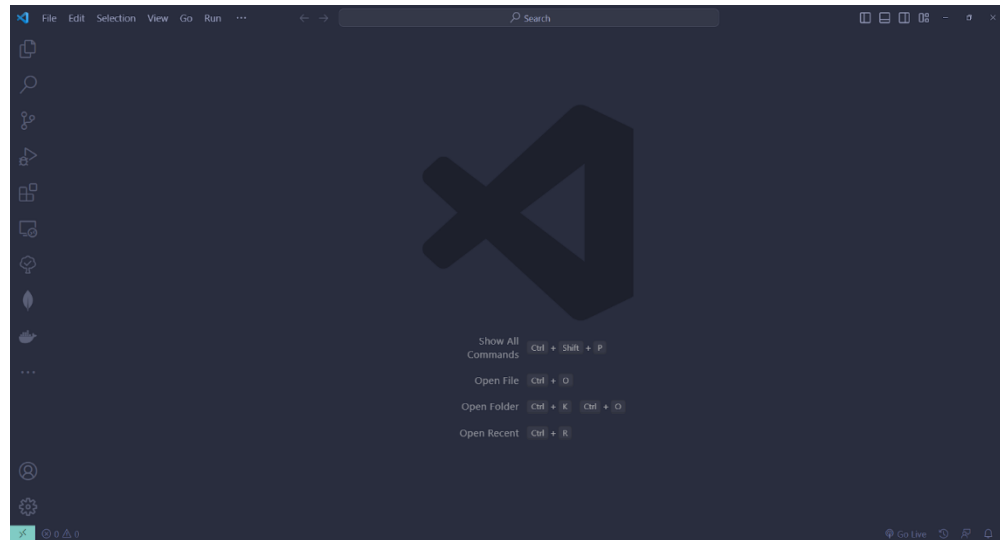
Selanjutnya ada Firebase Hosting, suatu layanan hosting konten web. Hanya dengan satu instruksi, kamu dapat mengimplementasikan aplikasi web serta menyajikan konten statis maupun dinamis ke CDN (jaringan penayangan konten) global dengan cepat.

Kegunaan dari Firebase Hosting itu sendiri yaitu mampu menayangkan konten melalui koneksi yang begitu aman, mengirimkan konten secara cepat, dan mendukung semua jenis konten untuk di hosting, mulai dari file HTML dan CSS hingga API atau layanan mikro Express.js.

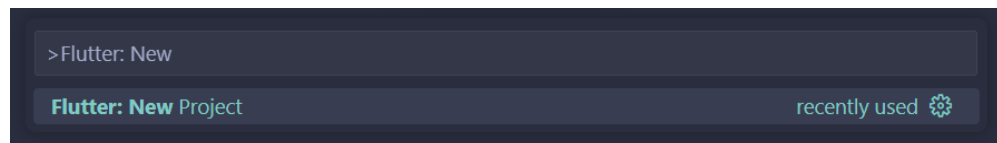
2. Membuat Aplikasi Flutter CRUD

Seperti sebelumnya melakukan init aplikasi flutter bisa menggunakan vscode, dan berikut langkah-langkah nya:

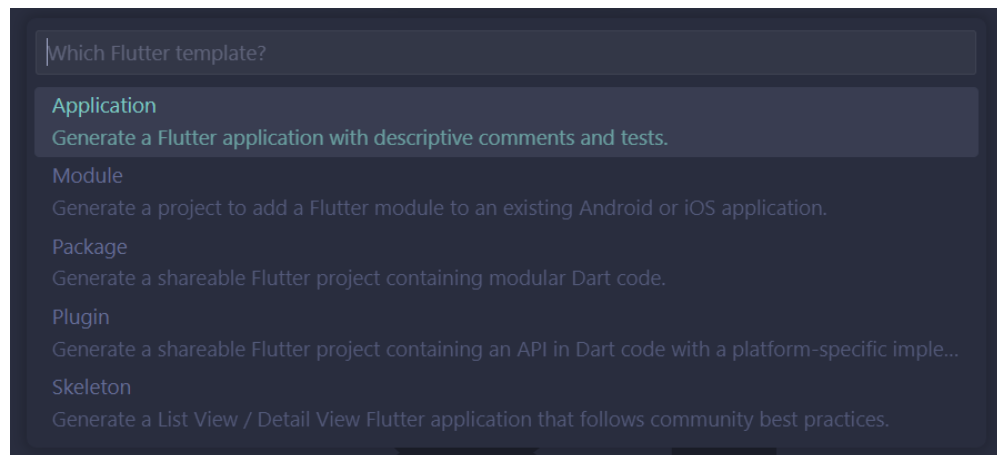
a. Jalankan VS Code



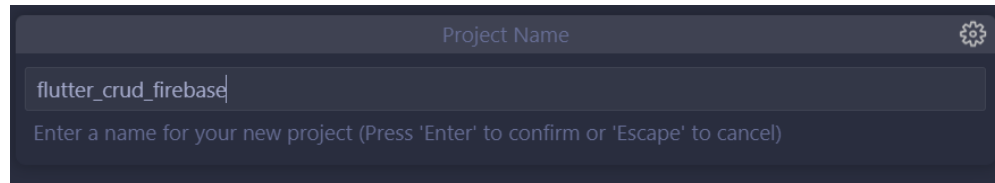
b. Ketik CTRL + SHIFT + P kemudian ketik dan pilih 'Flutter: New Project'.



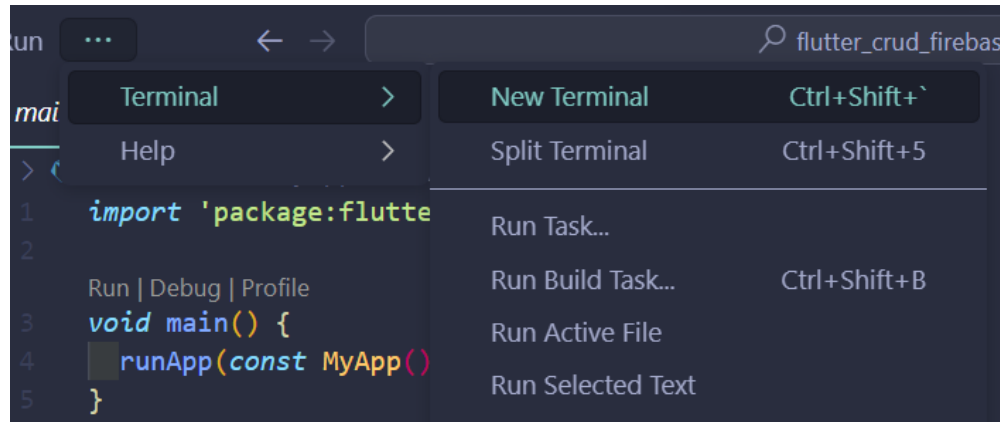
c. Pilih Application, lalu pilih lokasi penyimpanan project yang akan digunakan.



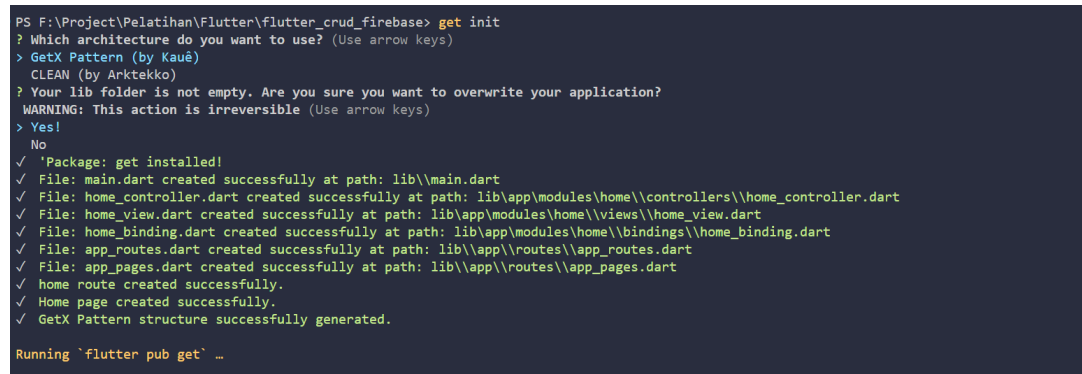
d. Berikan nama project, disini penulis memberikan nama 'flutter_crud_firebase'.



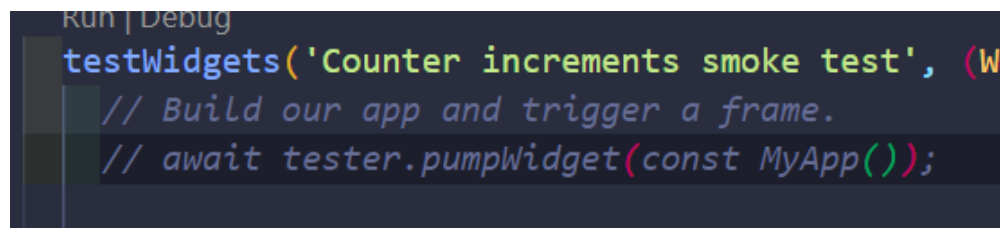
- e. Selanjutnya jalankan perintah 'get init' pada terminal VS Code



- f. Selanjutnya pilih 'GetX Pattern' dan pilih 'Yes', maka akan menjadi seperti berikut ini:



- g. Setelah selesai selanjutnya bisa menonaktifkan kode berikut ini pada file 'test/widget_test.dart'



- b. Instal alat command line yang diperlukan

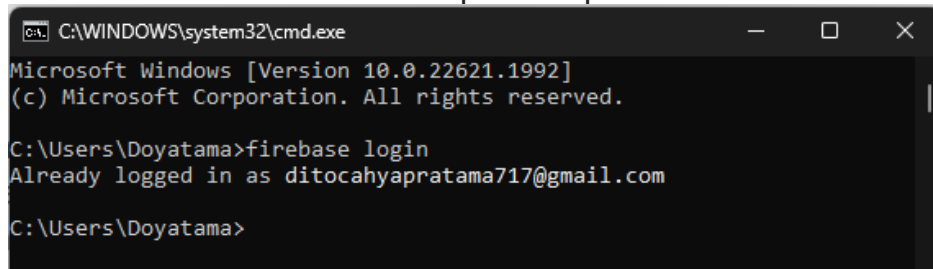
- Instal Firebase CLI jika Anda belum melakukannya.
- Instal [Node.js](#) menggunakan [nvm-windows](#) (Pengelola Versi Node). Menginstal Node.js secara otomatis juga akan menginstal alat perintah [npm](#).
- Instal Firebase CLI melalui [npm](#) dengan menjalankan perintah berikut:

```
npm install -g firebase-tools
```

- Login ke Firebase menggunakan akun Google Anda dengan menjalankan perintah berikut:

```
firebase login
```

Jika berhasil maka akan menampilkan seperti berikut ini:



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.22621.1992]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Doyatama>firebase login
Already logged in as ditocahyapratama717@gmail.com

C:\Users\Doyatama>
```

- Uji untuk memastikan bahwa CLI sudah diinstal dan mengakses akun Anda dengan benar dengan menampilkan daftar project Firebase Anda. Jalankan perintah berikut:

```
firebase projects:list
```

Jika berhasil maka akan menampilkan seperti berikut ini:

C:\WINDOWS\system32\cmd.exe

C:\Users\Doyatama>firebase projects:list
 ✓ Preparing the list of your Firebase projects

Project Display Name	Project ID	Project Number	Resource Location ID
aplikasi pk1	aplikasi-pk1-1f30c	92660138835	us-central
attendance	attendance-dd7fc	132196172269	asia-south1
connectyCubeTest	connectycubetest-e03dc	938755820371	[Not specified]
firecommerce	firecommerce-e359b	168091659000	us-central
flutter-tracking	flutter-tracking-b1816	878893311997	asia-southeast2
football	football-3ce1f	484197386555	[Not specified]
football-pwa	football-pwa-7ab87	795484955527	[Not specified]
Gas Online	gas-online-11e56	307301326022	us-central
get-cashier	get-cashier	1052852948218	us-central
Get School	get-school-bba83	597432629729	[Not specified]
guru mengajar	guru-mengajar-c1e82	718974786215	us-central
indiana-jones	indiana-jones-8e0bc	63040333481	us-central

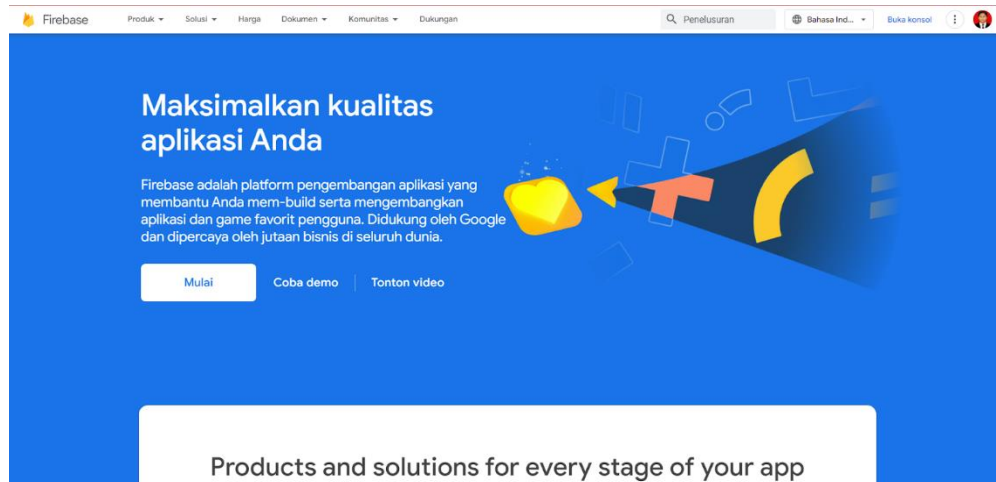
- Login ke Firebase menggunakan akun Google Anda dengan menjalankan perintah berikut:

```
firebase login
```

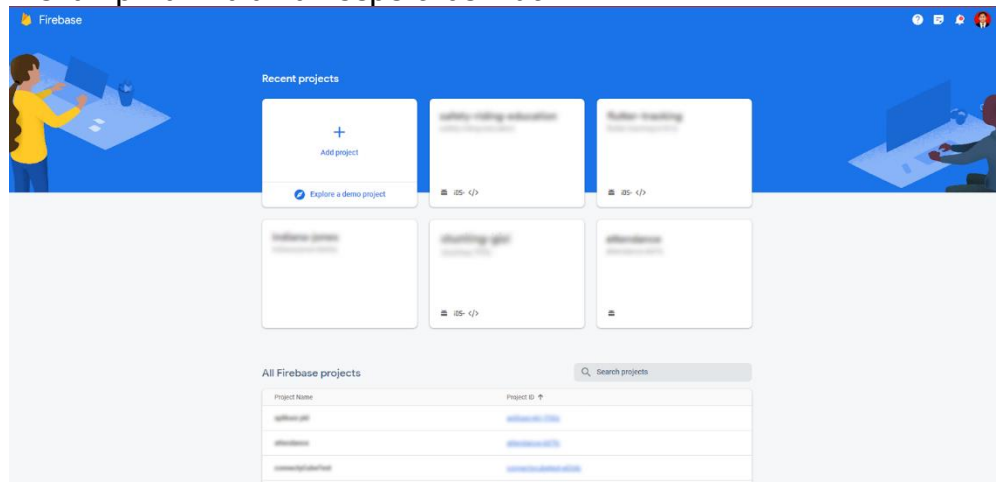
- Instal Flutter Fire CLI dengan menjalankan perintah berikut dari direktori flutter_crud_firebase:

```
dart pub global activate flutterfire_cli
```

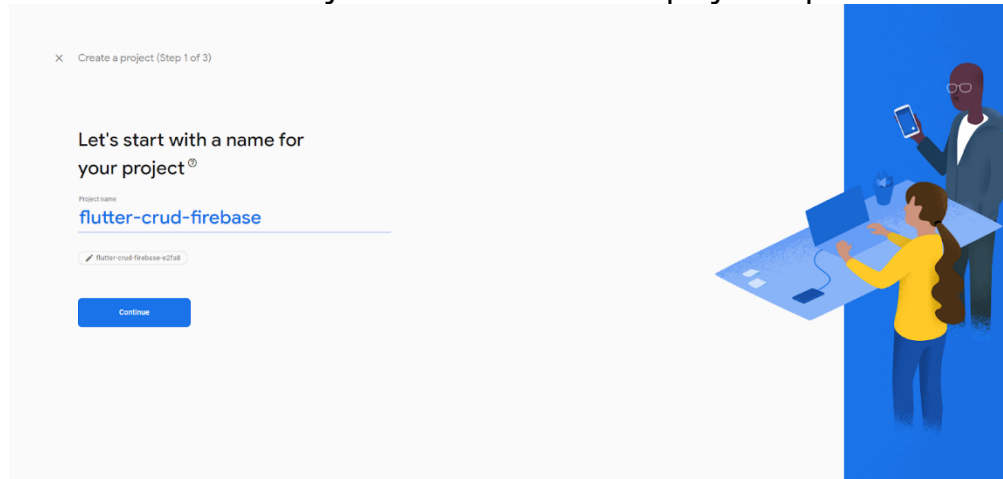
- Konfigurasi aplikasi Anda untuk menggunakan Firebase
 Gunakan FlutterFire CLI untuk mengonfigurasi aplikasi Flutter agar dapat terhubung ke Firebase.
- Membuat firebase di firebase console dengan cara mengakses halaman berikut ini: <https://firebase.google.com/>



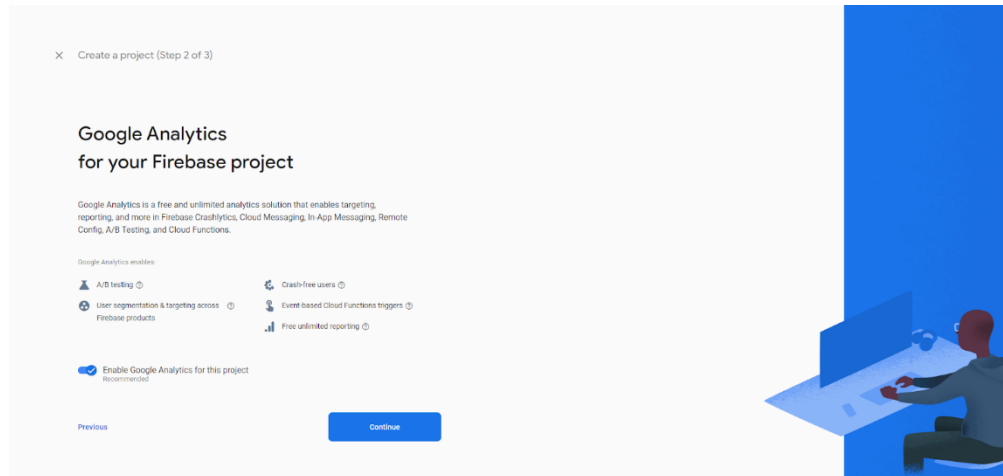
- Selanjutnya klik tombol buka konsol pada pojok kanan atas, lalu akan menampilkan halaman seperti berikut ini:



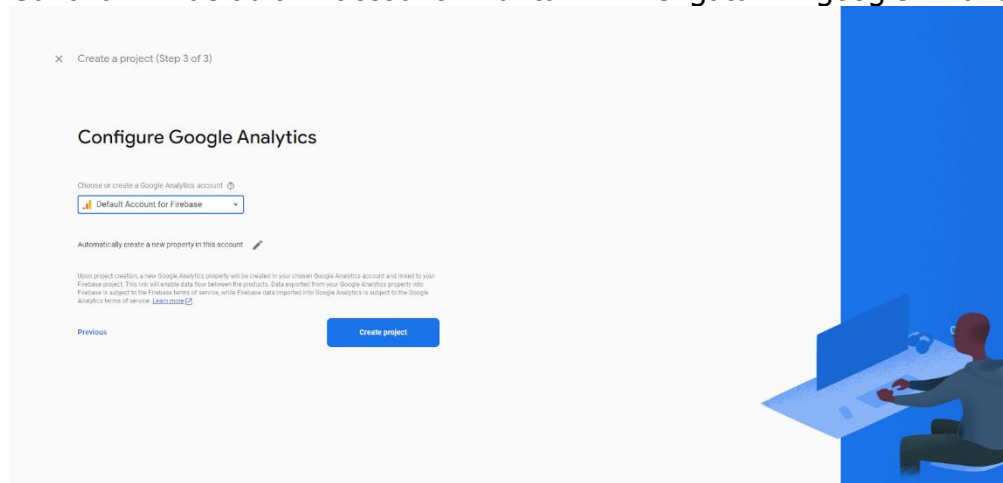
- Kemudian klik Add Project dan ketikkan nama project seperti berikut ini:



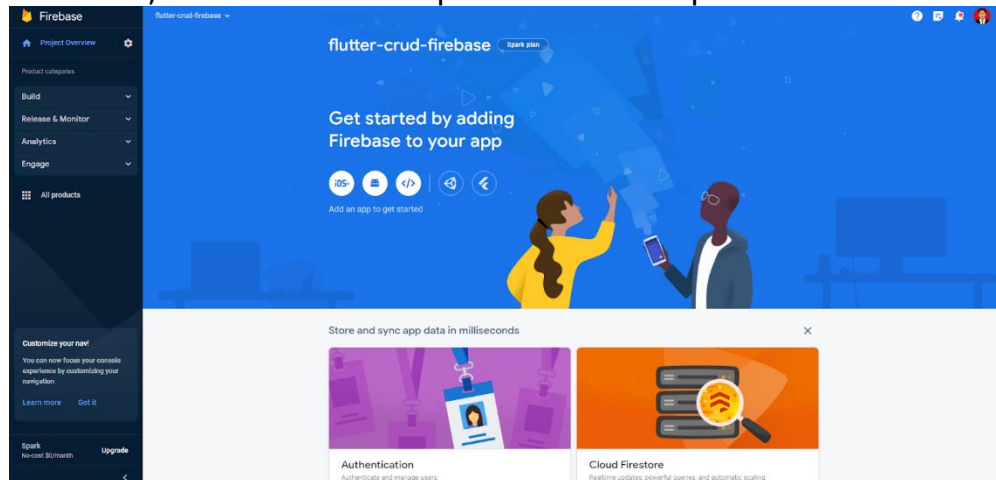
- Pada langkah selanjutnya mengaktifkan google analytic untuk project firebasenya, lalu klik 'Continue':



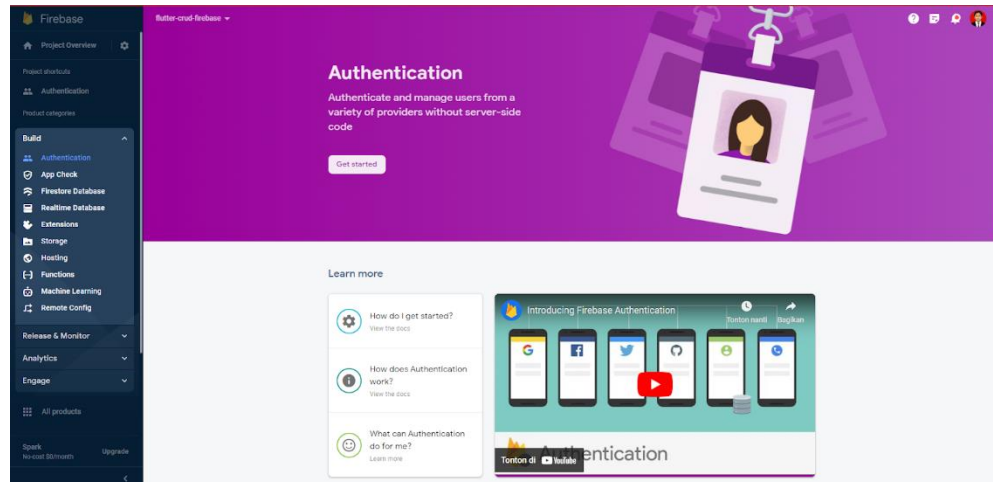
- Gunakan default account untuk mengatur google analytic



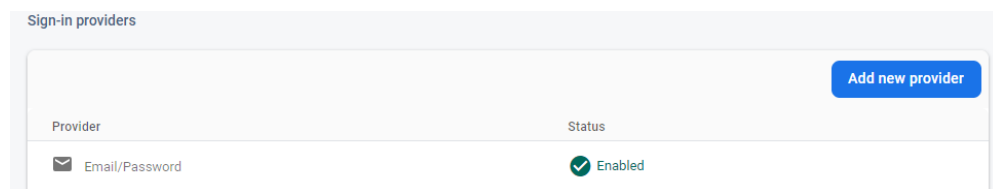
- Jika sudah, tunggu hingga proses membuat firebase project selesai lalu klik 'Continue', maka akan menampilkan halaman seperti berikut ini:



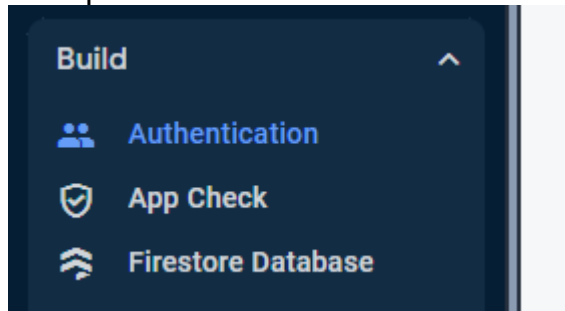
- Pilih Authentication pada menu Build, yang akan digunakan sebagai auth. Lalu pilih Get Started



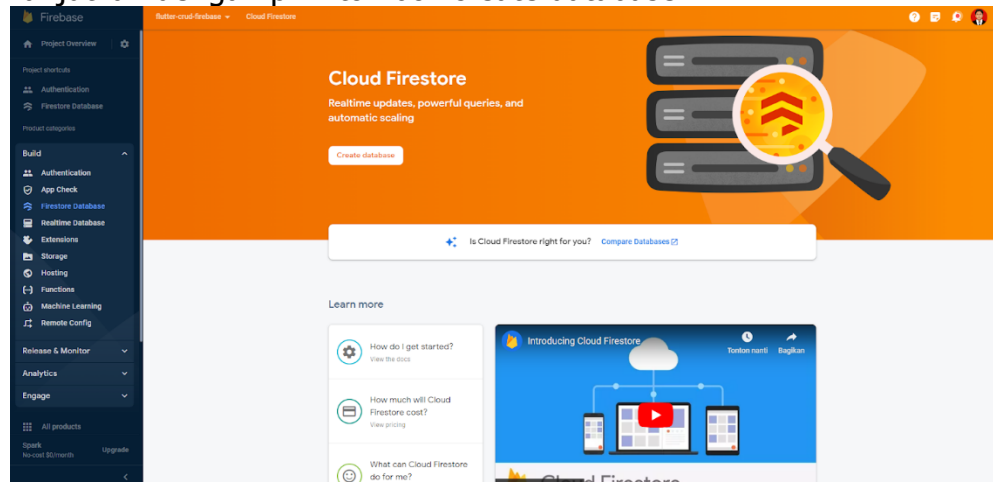
- Jangan lupa untuk mengaktifkan email/password sign in.



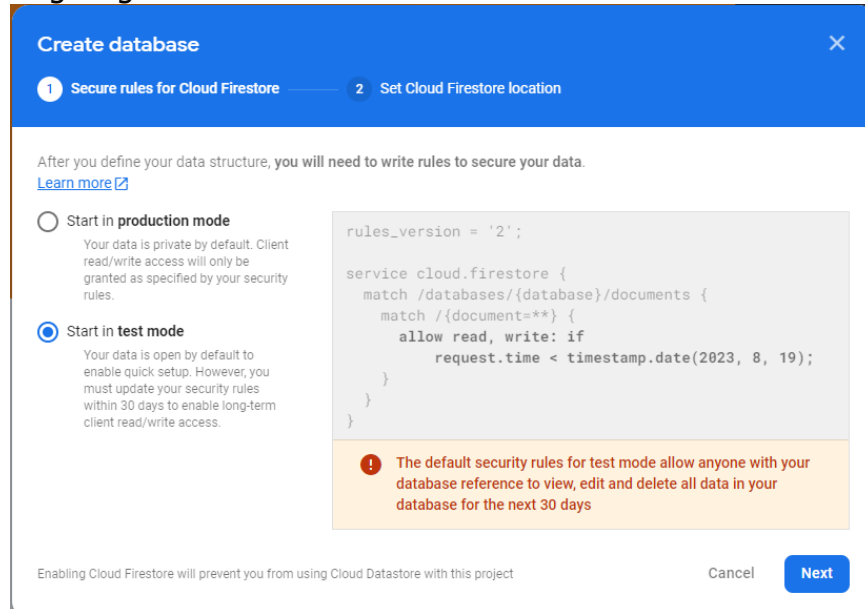
- Dan jika sudah selanjutnya mengaktifkan firestore database, dengan cara klik pada menu build sebelah kiri dan pilih Firestore Database.



- Lanjutkan dengan pilih tombol 'create database'.



- Pilih yang test mode lalu pilih next, begitu juga untuk location silahkan langsung diklik tombol next.



- Dari direktori project Flutter Anda, jalankan perintah berikut untuk memulai alur kerja konfigurasi aplikasi:

```
flutterfire configure
```

Pilih firebase project yang sudah dibuat sebelumnya,

```
connectycubetest-e03dc (connectycubetest)
firecommerce-e359b (firecommerce)
> flutter-crud-firebase-e2fa8 (flutter-crud-firebase)
flutter-tracking-b1816 (flutter-tracking)
football-3ce1f (football)
football-pwa-7ab87 (football-pwa)
gas-online-11e56 (Gas Online)
get-cashier (get-cashier)
```

Centang semua yang ada di pilihan pengaturan aplikasi flutter untuk firebase dengan menggunakan 'space'.

```
✓ Select a Firebase project to configure your Flutter application with · flutter-crud-firebase-e2fa8 (flutter-crud-firebase)
? Which platforms should your configuration support (use arrow keys & space to select)?
✓ android
✓ ios
✓ macos
✓ web
```

Tunggu hingga proses sinkronisasi antara aplikasi flutter dengan firebase selesai.

```

✓ Select a Firebase project to configure your Flutter application with · flutter-crud-firebase-e2fa8 (flutter-crud-firebase)
✓ Which platforms should your configuration support (use arrow keys & space to select)? · android, ios, macos, web
i Firebase android app com.example.flutter_crud_firebase is not registered on Firebase project flutter-crud-firebase-e2fa8.
i Registered a new Firebase android app on Firebase project flutter-crud-firebase-e2fa8.
i Firebase ios app com.example.flutterCrudFirebase is not registered on Firebase project flutter-crud-firebase-e2fa8.
i Registered a new Firebase ios app on Firebase project flutter-crud-firebase-e2fa8.
i Firebase macos app com.example.flutterCrudFirebase registered.
i Firebase web app flutter_crud_firebase (web) is not registered on Firebase project flutter-crud-firebase-e2fa8.
i Registered a new Firebase web app on Firebase project flutter-crud-firebase-e2fa8.

Firebase configuration file lib\firebase_options.dart generated successfully with the following Firebase apps:

Platform  Firebase App Id
web       1:551027975588:web:6977a48c5c04a2f473cd31
android   1:551027975588:android:5d2a9c5e7a8dee4973cd31
ios       1:551027975588:ios:a33a165b22fb58a273cd31
macos     1:551027975588:ios:a33a165b22fb58a273cd31

Learn more about using this file and next steps from the documentation:
> https://firebase.google.com/docs/flutter/setup
PS F:\Project\Pelatihan\Flutter\flutter_crud_firebase>

```

Apa fungsi alur kerja flutterfire configure ini?

- Meminta Anda memilih platform (iOS, Android, Web) yang didukung di aplikasi Flutter. Untuk setiap platform yang dipilih, FlutterFire CLI akan membuat aplikasi Firebase baru di project Firebase Anda. Anda dapat memilih untuk menggunakan project Firebase yang ada atau membuat project Firebase baru. Jika Anda sudah memiliki aplikasi yang terdaftar di project Firebase yang ada, FlutterFire CLI akan mencoba mencocokkannya berdasarkan konfigurasi project Flutter Anda saat ini.
 - Membuat file konfigurasi Firebase (firebase_options.dart) dan menambahkannya ke direktori lib/ Anda.
 - (untuk Crashlytics atau Performance Monitoring di Android) Tambahkan plugin Gradle khusus produk yang diperlukan ke aplikasi Flutter Anda.
- d. Lakukan inisialisasi Firebase di aplikasi Anda
- Dari direktori project Flutter Anda, jalankan perintah berikut untuk menginstal plugin inti:

```
flutter pub add firebase_core
```

- Dari direktori project Flutter Anda, jalankan perintah berikut untuk memastikan bahwa konfigurasi Firebase aplikasi Flutter Anda sudah yang terbaru:

```
flutterfire configure
```

```

✓ Select a Firebase project to configure your Flutter application with · flutter-crud-firebase-e2fa8 (flutter-crud-firebase)
✓ Which platforms should your configuration support (use arrow keys & space to select)? · android, ios, macos, web
i Firebase android app com.example.flutter_crud_firebase registered.
i Firebase ios app com.example.flutterCrudFirebase registered.
i Firebase macos app com.example.flutterCrudFirebase registered.
i Firebase web app flutter_crud_firebase (web) registered.

Firebase configuration file lib\firebase_options.dart generated successfully with the following Firebase apps:

Platform  Firebase App Id
web        1:551027975588:web:6977a48c5c04a2f473cd31
android    1:551027975588:android:5d2a9c5e7a8dee4973cd31
ios        1:551027975588:ios:a33a165b22fb58a273cd31
macos      1:551027975588:ios:a33a165b22fb58a273cd31

Learn more about using this file and next steps from the documentation:
> https://firebase.google.com/docs/flutter/setup
PS F:\Project\Pelatihan\Flutter\flutter_crud_firebase>

```

- Dalam file lib/main.dart, impor plugin inti Firebase dan file konfigurasi yang Anda buat sebelumnya:

```

import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

```

- Selain itu, di file lib/main.dart, lakukan inisialisasi Firebase menggunakan objek DefaultFirebaseOptions yang diekspor oleh file konfigurasi:

```

Future<void> main() async {
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  runApp(
    MaterialApp(
      title: "Application",
      initialRoute: AppPages.INITIAL,
      getPages: AppPages.routes,
    ),
  );
}

```

- Build ulang aplikasi Flutter Anda:

```

flutter run

```

e. Tambahkan plugin Firebase

Anda mengakses Firebase di aplikasi Flutter melalui berbagai plugin Firebase Flutter, satu plugin untuk setiap produk Firebase (misalnya: Cloud Firestore, Authentication, Analytics, dll.).

Karena Flutter adalah framework multi-platform, setiap plugin Firebase berlaku untuk platform Apple, Android, dan web. Jadi, jika Anda menambahkan plugin Firebase ke aplikasi Flutter, plugin tersebut akan digunakan oleh versi Apple, Android, dan web aplikasi Anda.

Berikut cara menambahkan plugin Firebase Flutter:

- Dari direktori project Flutter Anda, jalankan perintah berikut:

```
flutter pub add firebase_auth firebase_storage  
cloud_firestore
```

- Dari direktori project Flutter Anda, jalankan perintah berikut:

```
flutterfire configure
```

Dengan menjalankan perintah ini, Anda dapat memastikan konfigurasi Firebase aplikasi Flutter sudah yang terbaru dan dapat menambahkan plugin Gradle yang diperlukan ke aplikasi Anda untuk Crashlytics dan Performance Monitoring di Android.

- Setelah selesai, bangun ulang project Flutter Anda:

```
flutter run
```

- Jika berhasil maka akan menampilkan halaman seperti berikut ini:



4. Mengatur Utilitas Warna Aplikasi

- Buatlah file dengan nama `app_color.dart` pada folder `lib/app/utils`. Pada file ini akan kita gunakan sebagai master warna yang akan digunakan dalam pembuatan aplikasi.
- Berikut source code nya:

```
class AppColor {
  static LinearGradient primaryGradient = LinearGradient(
    colors: [primary, primarySoft],
    begin: Alignment.topCenter,
    end: Alignment.bottomCenter,
  );
  static Color primary = Color(0xFF2167E6);
  static Color primarySoft = Color(0xFF184AA6);
  static Color primaryExtraSoft = Color(0xFFEFF3FC);
  static Color secondary = Color(0xFF1B1F24);
  static Color secondarySoft = Color(0xFF9D9D9D);
  static Color secondaryExtraSoft = Color(0xFFE9E9E9);
  static Color error = Color(0xFFD00E0E);
  static Color success = Color(0xFF16AE26);
  static Color warning = Color(0xFFEB8600);
}
```

5. Membuat Custom Widget

- Supaya memudahkan dalam pembuatan component yang berulang kali, kita bisa memanfaatkan widget. untuk itu disini akan ada 3 widget yang sering digunakan berkali-kali diantaranya: `custom_input.dart`, `custom_toast.dart`, `custom_alert_dialog.dart`.
- `custom_input.dart` digunakan untuk memberikan style pada input sehingga lebih bagus untuk UI nya. Silahkan buat file dengan nama `custom_input.dart` pada folder `app/widgets`.

```
class CustomInput extends StatefulWidget {
  final TextEditingController controller;
```

```

final String label;
final String hint;
final bool disabled;
final EdgeInsetsGeometry margin;
final bool obscureText;
final Widget? suffixIcon;
final bool isDate;
final bool isNumber;
final bool isClickEmpty;
CustomInput({
  required this.controller,
  required this.label,
  required this.hint,
  this.disabled = false,
  this.margin = const EdgeInsets.only(bottom: 16),
  this.obscureText = false,
  this.isDate = false,
  this.suffixIcon,
  this.isNumber = false,
  this.isClickEmpty = false,
});

@override
State<CustomInput> createState() => _CustomInputState();
}

class _CustomInputState extends State<CustomInput> {
  @override
  Widget build(BuildContext context) {
    print("built");
    return Material(
      color: Colors.white,
      child: Container(
        width: MediaQuery.of(context).size.width,
        padding: EdgeInsets.only(left: 14, right: 14, top: 4),
        margin: widget.margin,
        decoration: BoxDecoration(
          color: (widget.disabled == false)
            ? Colors.transparent
            : AppColor.primaryExtraSoft,
          borderRadius: BorderRadius.circular(8),
          border: Border.all(width: 1, color:
AppColor.secondaryExtraSoft),
        ),
        child: TextField(
          readOnly: widget.disabled,
          obscureText: widget.obscureText,
          style: TextStyle(fontSize: 14, fontFamily: 'poppins'),
          maxLines: 1,
          controller: widget.controller,
          keyboardType:
            widget.isNumber ? TextInputType.number :
TextInputType.text,
          onTap: () async {
            if (widget.isDate) {

```

```

        DateTime? pickedDate = await showDatePicker(
            context: context,
            initialDate: DateTime.now(),
            firstDate: DateTime(
                1900), //DateTime.now() - not to allow to choose
before today.
                lastDate: DateTime.now(),
            );
        if (pickedDate != null) {
            print(
                pickedDate); //pickedDate output format => 2021-03-
10 00:00:00.000
            String formattedDate =
                DateFormat('dd-MM-yyyy').format(pickedDate);
            print(
                formattedDate); //formatted date output using intl
package => 2021-03-16
            //you can implement different kind of Date Format here
            according to your requirement

            setState(() {
                widget.controller.text =
                formattedDate; //set output date to TextField
value.
            });
        } else {
            print("Date is not selected");
        }
    }
    if (widget.isClickEmpty) {
        setState(() {
            widget.controller.text = "";
        });
    }
},
decoration: InputDecoration(
    suffixIcon: widget.suffixIcon ?? SizedBox(),
    label: Text(
        widget.label,
        style: TextStyle(
            color: AppColor.secondarySoft,
            fontSize: 14,
        ),
    ),
    floatingLabelBehavior: FloatingLabelBehavior.always,
    border: InputBorder.none,
    hintText: widget.hint,
    hintStyle: TextStyle(
        fontSize: 14,
        fontFamily: 'poppins',
        fontWeight: FontWeight.w500,
        color: AppColor.secondarySoft,
    ),
),
),
),
),

```

```

    ),
  );
}
}

```

- `custom_toast.dart`, digunakan untuk menampilkan pesan secara singkat, biasanya untuk untuk menggambarkan notifikasi sementara yang muncul di bagian bawah layar perangkat seluler pengguna. Ini mirip dengan pesan pop-up kecil yang memberikan informasi singkat atau pesan kepada pengguna.

Biasanya, toast digunakan untuk memberikan umpan balik singkat tentang tindakan yang sedang berlangsung dalam aplikasi, seperti memberitahu pengguna bahwa tindakan berhasil dilakukan, atau memberikan pesan kesalahan jika ada sesuatu yang tidak berjalan seperti yang diharapkan. Silahkan buat file dengan nama `custom_toast.dart` pada folder `app/widgets`.

```

class CustomToast {
  static errorToast(String? title, String? message) {
    Get.rawSnackbar(
      duration: Duration(seconds: 4),
      dismissDirection: DismissDirection.horizontal,
      messageText: Row(
        crossAxisAlignment: CrossAxisAlignment.center,
        children: [
          SvgPicture.asset(
            "assets/icons/info-circle.svg",
            color: Colors.white,
          ),
          SizedBox(width: 16),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Container(
                  margin: EdgeInsets.only(bottom: 4),
                  child: Text(
                    title ?? "error",
                    style: TextStyle(
                      color: Colors.white,
                      fontWeight: FontWeight.w500,
                      fontFamily: 'poppins',
                    ),
                  ),
                ),
                Text(
                  message ?? "Add your error message here",
                  style: TextStyle(color: Colors.white),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

```

```

        ],
      ),
    ),
  ],
),
backgroundColor: AppColor.error,
padding: EdgeInsets.symmetric(horizontal: 16, vertical: 10),
margin: EdgeInsets.symmetric(horizontal: 16),
borderRadius: 8,
snackPosition: SnackPosition.TOP,
);
}

static successToast(String? title, String? message) {
  Get.rawSnackbar(
    duration: Duration(seconds: 4),
    dismissDirection: DismissDirection.horizontal,
    messageText: Row(
      crossAxisAlignment: CrossAxisAlignment.center,
      children: [
        SvgPicture.asset(
          "assets/icons/success-circle.svg",
          color: Colors.white,
        ),
        SizedBox(width: 16),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Container(
                margin: EdgeInsets.only(bottom: 4),
                child: Text(
                  title ?? "Success",
                  style: TextStyle(
                    color: Colors.white,
                    fontWeight: FontWeight.w500,
                    fontFamily: 'poppins',
                  ),
                ),
              ),
              Text(
                message ?? "Add your success message here",
                style: TextStyle(color: Colors.white),
              ),
            ],
          ),
        ),
      ],
    ),
    backgroundColor: AppColor.success,
    padding: EdgeInsets.symmetric(horizontal: 16, vertical: 10),
    margin: EdgeInsets.symmetric(horizontal: 16),
    borderRadius: 8,
    snackPosition: SnackPosition.TOP,
  );
}

```

```
}  
}
```

- `custom_alert_dialog.dart`, merupakan widget bawaan Flutter yang digunakan untuk menampilkan jendela notifikasi atau peringatan kepada pengguna.

Saat Anda ingin menampilkan pesan atau informasi secara interaktif kepada pengguna dalam bentuk dialog, `AlertDialog` adalah salah satu pilihan yang bagus. Namun, ada situasi di mana tampilan bawaan `AlertDialog` mungkin tidak sesuai dengan desain atau kebutuhan aplikasi Anda. Di sinilah konsep "custom alert dialog" masuk; Anda dapat mengganti tampilan dan perilaku `AlertDialog` agar sesuai dengan kebutuhan Anda.

```
class CustomAlertDialog {  
  static confirmAdmin({  
    required String title,  
    required String message,  
    required void Function() onConfirm,  
    required void Function() onCancel,  
    required TextEditingController controller,  
  }) {  
    Get.defaultDialog(  
      title: "",  
      contentPadding: EdgeInsets.symmetric(horizontal: 24, vertical: 0),  
      radius: 8,  
      titlePadding: EdgeInsets.zero,  
      titleStyle: TextStyle(fontSize: 0),  
      content: Column(  
        children: [  
          Container(  
            margin: EdgeInsets.symmetric(vertical: 24),  
            child: Column(  
              crossAxisAlignment: CrossAxisAlignment.start,  
              children: [  
                Text(  
                  title,  
                  style: TextStyle(  
                    fontSize: 16,  
                    fontFamily: 'poppins',  
                    fontWeight: FontWeight.w600,  
                  ),  
                ),  
                SizedBox(height: 16),  
                Text(  
                  message,  
                  style: TextStyle(  
                    color: AppColor.secondarySoft,  
                    height: 150 / 100,  
                  ),  
                ),  
              ],  
            ),  
          ],  
        ),  
      ),  
    ),  
  },  
);
```

```

        ),
      ],
    ),
  ),
  CustomInput(
    margin: EdgeInsets.only(bottom: 24),
    controller: controller,
    label: 'password',
    hint: '*****',
    obscureText: true,
  ),
  Container(
    margin: EdgeInsets.only(bottom: 16),
    child: Row(
      children: [
        Expanded(
          flex: 6,
          child: ElevatedButton(
            onPressed: onCancel,
            child: Text(
              "batal",
              style: TextStyle(color: AppColor.secondarySoft),
            ),
            style: ElevatedButton.styleFrom(
              padding: EdgeInsets.symmetric(vertical: 12),
              primary: AppColor.primaryExtraSoft,
              elevation: 0,
              onPressed: AppColor.primary,
            ),
          ),
        ),
        SizedBox(width: 16),
        Expanded(
          flex: 6,
          child: ElevatedButton(
            onPressed: onConfirm,
            child: Text("Iya"),
            style: ElevatedButton.styleFrom(
              primary: AppColor.primary,
              padding: EdgeInsets.symmetric(vertical: 12),
              elevation: 0,
            ),
          ),
        ),
      ],
    ),
  ),
);
}

static showPresenceAlert({
  required String title,
  required String message,

```



```

required void Function() onConfirm,
required void Function() onCancel,
}) {
  Get.defaultDialog(
    title: "",
    contentPadding: EdgeInsets.symmetric(horizontal: 24, vertical: 0),
    radius: 8,
    titlePadding: EdgeInsets.zero,
    titleStyle: TextStyle(fontSize: 0),
    content: Column(
      children: [
        Container(
          margin: EdgeInsets.only(bottom: 32, top: 24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                title,
                style: TextStyle(
                  fontSize: 16,
                  fontFamily: 'poppins',
                  fontWeight: FontWeight.w600,
                ),
              ),
              SizedBox(height: 16),
              Text(
                message,
                style: TextStyle(
                  color: AppColor.secondarySoft,
                  height: 150 / 100,
                ),
              ),
            ],
          ),
        ),
        Container(
          margin: EdgeInsets.only(bottom: 16),
          child: Row(
            children: [
              Expanded(
                flex: 6,
                child: ElevatedButton(
                  onPressed: onCancel,
                  child: Text(
                    "Batal",
                    style: TextStyle(color: AppColor.secondarySoft),
                  ),
                  style: ElevatedButton.styleFrom(
                    padding: EdgeInsets.symmetric(vertical: 12),
                    primary: AppColor.primaryExtraSoft,
                    elevation: 0,
                    onPrimary: AppColor.primary,
                  ),
                ),
              ),
              SizedBox(width: 16),
            ],
          ),
        ),
      ],
    ),
  ),

```

```
Expanded(
  flex: 6,
  child: ElevatedButton(
    onPressed: onConfirm,
    child: Text("Iya"),
    style: ElevatedButton.styleFrom(
      primary: AppColor.primary,
      padding: EdgeInsets.symmetric(vertical: 12),
      elevation: 0,
    ),
  ),
),
),
),
],
),
),
],
),
),
);
}
}
```

6. Login di GetX

- Tambahkan package berikut ini supaya bisa menampilkan gambar svg, mengkonversi tanggal, dan membuat uuid untuk id masing-masing data.

```
flutter pub add flutter_svg intl uuid
```

- Ubah main.dart menjadi seperti berikut ini

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(
    StreamBuilder<User?>(
      stream: FirebaseAuth.instance.authStateChanges(),
      builder: (context, snapshot) {
        if (snapshot.connectionState ==
ConnectionState.waiting) {
          return MaterialApp(
            home: Scaffold(
              body: Center(
```

```

        child: CircularProgressIndicator(),
      ),
    ),
  );
}
return GetMaterialApp(
  title: "Application",
  debugShowCheckedModeBanner: false,
  initialRoute: snapshot.data != null ?
AppPages.INITIAL : Routes.LOGIN,
  getPages: AppPages.routes,
  theme: ThemeData(
    scaffoldBackgroundColor: Colors.white,
    fontFamily: 'inter',
  ),
);
},
),
);
}

```

Pada kode tersebut membuat sebuah logika jika belum login maka akan diarahkan ke halaman login dan jika sudah login maka akan diarahkan ke halaman home.

- Selanjutnya buat halaman login dengan menjalankan kode berikut ini di terminal.

```
get create page:login
```

Jika sudah maka pada folder lib/app/modules akan ada folder baru dengan nama login. Langkah selanjutnya adalah memberikan kode untuk file login_controller.dart seperti berikut ini:

Deklarasi variabel bisa dilakukan seperti berikut ini:

```

RxBool isLoading = false.obs;
RxBool obscureText = true.obs;
TextEditingController emailC = TextEditingController();
TextEditingController passC = TextEditingController();
FirebaseFirestore firestore = FirebaseFirestore.instance;
FirebaseAuth auth = FirebaseAuth.instance;

```

Selanjutnya kita membuat fungsi untuk login. Dengan memberikan beberapa pengecekan sebelum melakukan login kita bisa

mengimplementasikannya dengan menggunakan percabangan if else seperti kode berikut ini:

```
Future<void> login() async {
  if (emailC.text.isNotEmpty && passC.text.isNotEmpty) {
    isLoading.value = true;
    try {
      final credential = await auth.signInWithEmailAndPassword(
        email: emailC.text.trim(),
        password: passC.text,
      );

      if (credential.user != null) {
        String uid = auth.currentUser!.uid;
        DocumentSnapshot<Map<String, dynamic>> query =
          await firestore.collection("users").doc(uid).get();
        Get.offAllNamed(Routes.HOME);
      }
      isLoading.value = false;
    } on FirebaseAuthException catch (e) {
      isLoading.value = false;
      if (e.code == 'user-not-found') {
        CustomToast.errorToast("Error", "Akun tidak ditemukan");
      } else if (e.code == 'wrong-password') {
        CustomToast.errorToast("Error", "Kata sandi salah");
      }
    } catch (e) {
      CustomToast.errorToast("Error", "Error dikarenakan :
      ${e.toString()}");
    }
  } else {
    CustomToast.errorToast(
      "Error", "Anda harus mengisi email dan kata sandi");
  }
}
```

Dan berikut ini adalah full kode untuk LoginController:

```
class LoginController extends GetxController {
  RxBool isLoading = false.obs;
  RxBool obscureText = true.obs;
  TextEditingController emailC = TextEditingController();
  TextEditingController passC = TextEditingController();
  FirebaseFirestore firestore = FirebaseFirestore.instance;
  FirebaseAuth auth = FirebaseAuth.instance;

  Future<void> login() async {
    if (emailC.text.isNotEmpty && passC.text.isNotEmpty) {
      isLoading.value = true;
      try {
        final credential = await auth.signInWithEmailAndPassword(
          email: emailC.text.trim(),
          password: passC.text,
        );
      }
    }
  }
}
```

```

);

if (credential.user != null) {
  String uid = auth.currentUser!.uid;
  DocumentSnapshot<Map<String, dynamic>> query =
    await firestore.collection("users").doc(uid).get();
  Get.offAllNamed(Routes.HOME);
}
isLoading.value = false;
} on FirebaseAuthException catch (e) {
  isLoading.value = false;
  if (e.code == 'user-not-found') {
    CustomToast.errorToast("Error", "Akun tidak ditemukan");
  } else if (e.code == 'wrong-password') {
    CustomToast.errorToast("Error", "Kata sandi salah");
  }
} catch (e) {
  CustomToast.errorToast("Error", "Error dikarenakan :
  ${e.toString()}");
}
} else {
  CustomToast.errorToast(
    "Error", "Anda harus mengisi email dan kata sandi");
}
}
}
}

```

- Implementasi kode berikut untuk halaman login_view.dart

```

class LoginView extends GetView<LoginController> {
  const LoginView({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: AppColor.primary,
        elevation: 0.0,
        titleSpacing: 10.0,
        centerTitle: true,
        title: Text("Login"),
      ),
      backgroundColor: AppColor.primary,
      body: ListView(
        shrinkWrap: true,
        children: [
          Container(
            height: MediaQuery.of(context).size.height * 25 / 100,
            width: MediaQuery.of(context).size.width,
            padding: EdgeInsets.only(left: 32),
            decoration: BoxDecoration(
              gradient: AppColor.primaryGradient,
              image: DecorationImage(

```

```

        image: AssetImage('assets/images/pattern-1-1.png'),
        fit: BoxFit.cover,
      ),
    ),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          "TODO App",
          style: TextStyle(
            fontSize: 24,
            color: Colors.white,
            fontFamily: 'poppins',
            height: 150 / 100,
            fontWeight: FontWeight.w600,
          ),
        ),
        SizedBox(height: 10),
        Text(
          "by Hummasoft",
          style: TextStyle(color: Colors.white),
        ),
      ],
    ),
  ),
  Container(
    height: MediaQuery.of(context).size.height * 65 / 100,
    width: MediaQuery.of(context).size.width,
    color: Colors.white,
    padding: EdgeInsets.only(left: 20, right: 20, top: 36,
bottom: 84),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.start,
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Container(
          margin: EdgeInsets.only(bottom: 24),
          child: Text(
            'Masuk',
            style: TextStyle(
              fontSize: 18,
              fontFamily: 'poppins',
              fontWeight: FontWeight.w600,
            ),
          ),
        ),
        Container(
          width: MediaQuery.of(context).size.width,
          padding: EdgeInsets.only(left: 14, right: 14, top: 4),
          margin: EdgeInsets.only(bottom: 16),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(8),
            border: Border.all(
              width: 1, color: AppColor.secondaryExtraSoft),
          ),
        ),
      ],
    ),
  ),
),

```

```

        ),
        child: TextField(
          style: TextStyle(fontSize: 14, fontFamily:
'poppins'),
          maxLines: 1,
          controller: controller.emailC,
          keyboardType: TextInputType.emailAddress,
          decoration: InputDecoration(
            label: Text(
              "Email",
              style: TextStyle(
                color: AppColor.secondarySoft,
                fontSize: 14,
              ),
            ),
            floatingLabelBehavior:
FloatingLabelBehavior.always,
            border: InputBorder.none,
            hintText: "email@gmail.com",
            hintStyle: TextStyle(
              fontSize: 14,
              fontFamily: 'poppins',
              fontWeight: FontWeight.w500,
              color: AppColor.secondarySoft,
            ),
          ),
        ),
      ),
    ),
    Material(
      child: Container(
        width: MediaQuery.of(context).size.width,
        padding: EdgeInsets.only(left: 14, right: 14, top:
4),
        margin: EdgeInsets.only(bottom: 24),
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(8),
          border: Border.all(
            width: 1, color: AppColor.secondaryExtraSoft),
        ),
        child: Obx(
          () => TextField(
            style: TextStyle(fontSize: 14, fontFamily:
'poppins'),
            maxLines: 1,
            controller: controller.passC,
            obscureText: controller.obsecureText.value,
            decoration: InputDecoration(
              label: Text(
                "Kata Sandi",
                style: TextStyle(
                  color: AppColor.secondarySoft,
                  fontSize: 14,
                ),
              ),
              floatingLabelBehavior:
FloatingLabelBehavior.always,

```



```

        Container(
          width: MediaQuery.of(context).size.width,
          child: ElevatedButton(
            onPressed: () async {
              Get.toNamed(Routes.REGISTER);
            },
            child: Text(
              'Daftar',
              style: TextStyle(
                fontSize: 16,
                fontFamily: 'poppins',
                fontWeight: FontWeight.w500,
                color: AppColor.primary,
              ),
            ),
            style: ElevatedButton.styleFrom(
              padding: EdgeInsets.symmetric(vertical: 18),
              elevation: 0,
              primary: Colors.white,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(8),
                side: BorderSide(
                  color: AppColor
                    .primary, // Replace this with your
desired border color
                    width:
border width
                    1.0, // Replace this with your desired
                ),
              ),
            ),
          ),
        ),
      ],
    ),
  ),
],
),
);
}
}

```

- Jika saat run muncul error seperti ini,

```
BUILD FAILED in 18s
Running Gradle task 'assembleDebug'... 19.3s

Flutter Fix
The plugin cloud_firestore requires a higher Android SDK version.
Fix this issue by adding the following to the file
F:\Project\Pelatihan\Flutter\flutter_crud_firebase\android\app\build.gradle:
android {
  defaultConfig {
    minSdkVersion 19
  }
}

Note that your app won't be available to users running Android SDKs below 19.
Alternatively, try to find a version of this plugin that supports these lower versions of the Android SDK.
For more information, see: https://docs.flutter.dev/deployment/android#reviewing-the-build-configuration
```

Maka lakukan troubleshoot dengan cara merubah minSdkVersion pada file android/app/build.gradle menjadi 19.

- Selesai, silahkan di jalankan dengan menggunakan 'flutter run'.

7. Register di Flutter

- Buat halaman register dengan menjalankan kode berikut ini di terminal.

```
get create page:register
```

Jika sudah maka pada folder lib/app/modules akan ada folder baru dengan nama register. Langkah selanjutnya adalah memberikan kode untuk file register_controller.dart seperti berikut ini:

Deklarasi variabel bisa dilakukan seperti berikut ini:

```
RxBool isLoading = false.obs;
RxBool obscureText = true.obs;
TextEditingController nameC = TextEditingController();
TextEditingController emailC = TextEditingController();
TextEditingController passC = TextEditingController();
FirebaseAuth auth = FirebaseAuth.instance;
FirebaseFirestore firestore = FirebaseFirestore.instance;
```

Selanjutnya membuat fungsi register, dimana pada fungsi ini kita akan melakukan pendaftaran pengguna pada build Authentication dan build Firestore Database. Hal ini dilakukan karena Authentication hanya bisa menyimpan email dan kata sandi, sedangkan Firestore Database digunakan untuk menyimpan teks.

```
Future<void> registration() async {
  if (passC.text.isNotEmpty) {
    isLoading.value = true;
    try {
```

```

        // if the password is match, then continue the create user
process
        UserCredential respondentCredential =
            await auth.createUserWithEmailAndPassword(
                email: emailC.text.trim(),
                password: passC.text,
            );

        if (respondentCredential.user != null) {
            String uid = respondentCredential.user!.uid;

            DocumentReference users =
                firestore.collection("users").doc(uid);
            await users.set({
                "user_id": uid,
                "name": nameC.text,
                "email": emailC.text.trim(),
                "created_at": DateTime.now().toIso8601String(),
            });

            await respondentCredential.user!.sendEmailVerification();
            //need to logout because the current user is changed after
adding new user
            auth.signOut();
            // need to relogin to admin account
            await auth.signInWithEmailAndPassword(
                email: emailC.text.trim(), password: passC.text);

            Get.back(); //close dialog
            Get.back(); //close form screen
            CustomToast.successToast('Success', 'berhasil mendaftar');

            isLoading.value = false;
        }
    } on FirebaseAuthException catch (e) {
        isLoading.value = false;
        if (e.code == 'weak-password') {
            CustomToast.errorToast('Error', 'kata sandi terlalu pendek');
        } else if (e.code == 'email-already-in-use') {
            CustomToast.errorToast('Error', 'Email sudah ada');
        } else if (e.code == 'wrong-password') {
            CustomToast.errorToast('Error', 'Salah kata sandi');
        } else {
            CustomToast.errorToast('Error', 'error : ${e.code}');
        }
    } catch (e) {
        isLoading.value = false;
        CustomToast.errorToast('Error', 'error : ${e.toString()}');
    }
    } else {
        CustomToast.errorToast('Error', 'Kata sandi harus diisikan');
    }
}
}

```

Dan berikut ini adalah full kode untuk RegisterController:

```
class RegisterController extends GetxController {
    RxBool isLoading = false.obs;
    RxBool obscureText = true.obs;
    TextEditingController nameC = TextEditingController();
    TextEditingController emailC = TextEditingController();
    TextEditingController passC = TextEditingController();
    FirebaseAuth auth = FirebaseAuth.instance;
    FirebaseFirestore firestore = FirebaseFirestore.instance;

    @override
    Future<void> onInit() async {
        super.onInit();
    }

    Future<void> registration() async {
        if (passC.text.isNotEmpty) {
            isLoading.value = true;
            try {
                // if the password is match, then continue the create user process
                UserCredential respondentCredential =
                    await auth.createUserWithEmailAndPassword(
                        email: emailC.text.trim(),
                        password: passC.text,
                    );

                if (respondentCredential.user != null) {
                    String uid = respondentCredential.user!.uid;

                    DocumentReference users = firestore.collection("users").doc(uid);
                    await users.set({
                        "user_id": uid,
                        "name": nameC.text,
                        "email": emailC.text.trim(),
                        "created_at": DateTime.now().toIso8601String(),
                    });

                    await respondentCredential.user!.sendEmailVerification();
                    //need to logout because the current user is changed after adding
                    new user
                    auth.signOut();
                    // need to relogin to admin account
                    await auth.signInWithEmailAndPassword(
                        email: emailC.text.trim(), password: passC.text);

                    Get.back(); //close dialog
                    Get.back(); //close form screen
                    CustomToast.successToast('Success', 'berhasil mendaftar');

                    isLoading.value = false;
                }
            } on FirebaseAuthException catch (e) {
                isLoading.value = false;
            }
        }
    }
}
```

```

    if (e.code == 'weak-password') {
      CustomToast.errorToast('Error', 'kata sandi terlalu pendek');
    } else if (e.code == 'email-already-in-use') {
      CustomToast.errorToast('Error', 'Email sudah ada');
    } else if (e.code == 'wrong-password') {
      CustomToast.errorToast('Error', 'Salah kata sandi');
    } else {
      CustomToast.errorToast('Error', 'error : ${e.code}');
    }
  } catch (e) {
    isLoading.value = false;
    CustomToast.errorToast('Error', 'error : ${e.toString()}');
  }
} else {
  CustomToast.errorToast('Error', 'Kata sandi harus diisi');
}
}
}

```

- Implementasi kode berikut untuk halaman register_view.dart

```

class RegisterView extends GetView<RegisterController> {
  const RegisterView({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppColor.primary,
      appBar: AppBar(
        automaticallyImplyLeading: true,
        elevation: 0.0,
        titleSpacing: 10.0,
        centerTitle: true,
        title: Text("Pendaftaran"),
        leading: IconButton(
          icon: SvgPicture.asset(
            'assets/icons/arrow-left.svg',
            color: Colors.white,
          ),
          onPressed: () => Get.back(),
        ),
        backgroundColor: Colors.transparent,
      ),
      extendBodyBehindAppBar: true,
      body: ListView(
        shrinkWrap: true,
        children: [
          Container(
            height: MediaQuery.of(context).size.height * 25 / 100,
            width: MediaQuery.of(context).size.width,
            padding: EdgeInsets.only(left: 32),
            decoration: BoxDecoration(
              gradient: AppColor.primaryGradient,
              image: DecorationImage(

```

```

        image: AssetImage('assets/images/pattern-1-1.png'),
        fit: BoxFit.cover,
      ),
    ),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          "TODO App",
          style: TextStyle(
            fontSize: 24,
            color: Colors.white,
            fontFamily: 'poppins',
            height: 150 / 100,
            fontWeight: FontWeight.w600,
          ),
        ),
        SizedBox(height: 10),
        Text(
          "by Hummasoft",
          style: TextStyle(color: Colors.white),
        ),
      ],
    ),
  ),
  Container(
    height: MediaQuery.of(context).size.height * 70 / 100,
    width: MediaQuery.of(context).size.width,
    color: Colors.white,
    padding: EdgeInsets.only(left: 20, right: 20, top: 36,
bottom: 84),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.start,
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Container(
          margin: EdgeInsets.only(bottom: 24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'Daftar',
                style: TextStyle(
                  fontSize: 18,
                  fontFamily: 'poppins',
                  fontWeight: FontWeight.w600,
                ),
              ),
            ],
          ),
        ),
        Container(
          width: MediaQuery.of(context).size.width,
          padding: EdgeInsets.only(left: 14, right: 14, top: 4),
          margin: EdgeInsets.only(bottom: 16),

```

```

        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius: BorderRadius.circular(8),
          border: Border.all(
            width: 1, color: AppColor.secondaryExtraSoft),
        ),
        child: TextField(
          style: TextStyle(fontSize: 14, fontFamily:
'poppins'),
          maxLines: 1,
          controller: controller.nameC,
          decoration: InputDecoration(
            label: Text(
              "Nama Lengkap",
              style: TextStyle(
                color: AppColor.secondarySoft,
                fontSize: 14,
              ),
            ),
            floatingLabelBehavior:
FloatingLabelBehavior.always,
            border: InputBorder.none,
            hintText: "nama lengkap anda",
            hintStyle: TextStyle(
              fontSize: 14,
              fontFamily: 'poppins',
              fontWeight: FontWeight.w500,
              color: AppColor.secondarySoft,
            ),
          ),
        ),
      ),
    ),
    Container(
      width: MediaQuery.of(context).size.width,
      padding: EdgeInsets.only(left: 14, right: 14, top: 4),
      margin: EdgeInsets.only(bottom: 16),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(8),
        border: Border.all(
          width: 1, color: AppColor.secondaryExtraSoft),
      ),
      child: TextField(
        style: TextStyle(fontSize: 14, fontFamily:
'poppins'),
        maxLines: 1,
        controller: controller.emailC,
        decoration: InputDecoration(
          label: Text(
            "Email",
            style: TextStyle(
              color: AppColor.secondarySoft,
              fontSize: 14,
            ),
          ),
        ),
      ),
    ),
  ),
)

```

```

        floatingLabelBehavior:
FloatingLabelBehavior.always,
        border: InputBorder.none,
        hintText: "email anda",
        hintStyle: TextStyle(
            fontSize: 14,
            fontFamily: 'poppins',
            fontWeight: FontWeight.w500,
            color: AppColor.secondarySoft,
        ),
    ),
),
),
),
Container(
    margin: EdgeInsets.only(bottom: 24),
    child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
            Material(
                child: Container(
                    width: MediaQuery.of(context).size.width,
                    padding: EdgeInsets.only(left: 14, right: 14,
top: 4),

                    decoration: BoxDecoration(
                        borderRadius: BorderRadius.circular(8),
                        border: Border.all(
                            width: 1, color:
AppColor.secondaryExtraSoft),
                    ),
                    child: Obx(
                        () => TextField(
                            style: TextStyle(
                                fontSize: 14, fontFamily: 'poppins'),
                            maxLines: 1,
                            controller: controller.passC,
                            obscureText: controller.obsecureText.value,
                            decoration: InputDecoration(
                                label: Text(
                                    "Kata Sandi",
                                    style: TextStyle(
                                        color: AppColor.secondarySoft,
                                        fontSize: 14,
                                    ),
                                ),
                            ),
                            floatingLabelBehavior:
                                FloatingLabelBehavior.always,
                            border: InputBorder.none,
                            hintText: "*****",
                            suffixIcon: IconButton(
                                icon: (controller.obsecureText !=
false)

                                ? SvgPicture.asset(
                                    'assets/icons/show.svg')
                                : SvgPicture.asset(
                                    'assets/icons/hide.svg'),
                                onPressed: () {

```



```

        controller.obsecureText.value =
            !(controller.obsecureText.value);
    },
    ),
    hintStyle: TextStyle(
        fontSize: 14,
        fontFamily: 'poppins',
        fontWeight: FontWeight.w500,
        color: AppColor.secondarySoft,
    ),
    ),
    ),
    ),
    ),
    ),
    Text(
        "Kata sandi minimal 6 karakter",
        style: TextStyle(
            color: AppColor.secondarySoft,
            height: 150 / 100,
        ),
    ),
    ],
    ),
    ),
    Obx(
        () => Container(
            width: MediaQuery.of(context).size.width,
            child: ElevatedButton(
                onPressed: () async {
                    if (controller.isLoading.isFalse) {
                        await controller.registration();
                    }
                },
                child: Text(
                    (controller.isLoading.isFalse)
                        ? 'Daftar'
                        : 'Loading...',
                    style: TextStyle(
                        fontSize: 16,
                        fontFamily: 'poppins',
                        fontWeight: FontWeight.w500,
                    ),
                ),
                style: ElevatedButton.styleFrom(
                    padding: EdgeInsets.symmetric(vertical: 18),
                    elevation: 0,
                    primary: AppColor.primary,
                    shape: RoundedRectangleBorder(
                        borderRadius: BorderRadius.circular(8),
                    ),
                ),
            ),
        ),
    ),
    ],

```

```

    ),
  ),
],
),
);
}
}

```

- Selesai, silahkan di jalankan dengan menggunakan 'flutter run'.

8. *Homepage* di Flutter

- Karena sebelumnya kita sudah berhasil membuat halaman untuk login dan register, untuk itu kita juga perlu mengimplementasi design dan controller untuk halaman homepage.
- Buat halaman home_controller.dart menjadi seperti berikut ini:

Melakukan deklarasi variabel terlebih dahulu

```

RxBool isLoading = false.obs;
FirebaseAuth auth = FirebaseAuth.instance;
FirebaseFirestore firestore = FirebaseFirestore.instance;

```

Selanjutnya membuat suatu fungsi stream pengguna yang digunakan untuk mengambil data pengguna dari Firestore Database, sehingga data yang ditampilkan selalu dalam keadaan realtime.

```

Stream<DocumentSnapshot<Map<String, dynamic>>> streamUser() async* {
  String uid = auth.currentUser!.uid;
  yield* firestore.collection("users").doc(uid).snapshots();
}

```

Karena dibutuhkan untuk perhitungan jumlah todo yang ada maka digunakan fungsi stream todo yang fungsinya mirip dengan stream pengguna.

```

Stream<QuerySnapshot<Map<String, dynamic>>> streamTodo() async* {
  String uid = auth.currentUser!.uid;
  yield* firestore
    .collection("users")
    .doc(uid)
    .collection("todos")
    .snapshots();
}

```

Untuk menampilkan data todo yang terbaru bisa menggunakan fungsi seperti berikut ini.

```
Stream<QuerySnapshot<Map<String, dynamic>>> streamLastTodo() async* {
  String uid = auth.currentUser!.uid;
  yield* firestore
    .collection("users")
    .doc(uid)
    .collection("todos")
    .orderBy("created_at", descending: true)
    .limitToLast(5)
    .snapshots();
}
```

Dan yang terakhir adalah fungsi logout, yang digunakan untuk pengguna yang ingin keluar dari aplikasi dan login dengan menggunakan akun yang lain.

```
void logout() async {
  CustomAlertDialog.showPresenceAlert(
    title: "Logout System",
    message: "Apakah anda yakin akan logout dari sistem ?",
    onCancel: () => Get.back(),
    onConfirm: () async {
      Get.back(); // close modal
      Get.back(); // back page
      try {
        await auth.signOut();
        Get.offAllNamed(Routes.LOGIN);
        CustomToast.successToast('Success', 'Berhasil Logout');
      } catch (e) {
        CustomToast.errorToast(
          "Error", "Error dikarenakan : ${e.toString()}");
      }
    },
  );
}
```

Berikut kode lengkap dari home_controller.dart

```
class HomeController extends GetxController {
  RxBool isLoading = false.obs;
  FirebaseAuth auth = FirebaseAuth.instance;
  FirebaseFirestore firestore = FirebaseFirestore.instance;

  Stream<DocumentSnapshot<Map<String, dynamic>>> streamUser() async* {
    String uid = auth.currentUser!.uid;
    yield* firestore.collection("users").doc(uid).snapshots();
  }
}
```

```

Stream<QuerySnapshot<Map<String, dynamic>>> streamTodo() async* {
  String uid = auth.currentUser!.uid;
  yield* firestore
    .collection("users")
    .doc(uid)
    .collection("todos")
    .snapshots();
}

Stream<QuerySnapshot<Map<String, dynamic>>> streamLastTodo() async* {
  String uid = auth.currentUser!.uid;
  yield* firestore
    .collection("users")
    .doc(uid)
    .collection("todos")
    .orderBy("created_at", descending: true)
    .limitToLast(5)
    .snapshots();
}

void logout() async {
  CustomAlertDialog.showPresenceAlert(
    title: "Logout System",
    message: "Apakah anda yakin akan logout dari sistem ?",
    onCancel: () => Get.back(),
    onConfirm: () async {
      Get.back(); // close modal
      Get.back(); // back page
      try {
        await auth.signOut();
        Get.offAllNamed(Routes.LOGIN);
        CustomToast.successToast('Success', 'Berhasil Logout');
      } catch (e) {
        CustomToast.errorToast(
          "Error", "Error dikarenakan : ${e.toString()}");
      }
    },
  );
}
}

```

- Buat halaman home_view.dart menjadi seperti berikut ini

```

import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/material.dart';
import
'package:flutter_crud_firebase/app/modules/home/controllers/home_controller.
dart';
import 'package:flutter_crud_firebase/app/routes/app_pages.dart';
import 'package:flutter_crud_firebase/app/utills/app_color.dart';
import 'package:get/get.dart';

```

```

class HomeView extends GetView<HomeController> {
  const HomeView({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      extendBody: true,
      body: StreamBuilder<DocumentSnapshot<Map<String, dynamic>>>(
        stream: controller.streamUser(),
        builder: (context, snapshot) {
          switch (snapshot.connectionState) {
            case ConnectionState.active:
            case ConnectionState.done:
              Map<String, dynamic> user = snapshot.data!.data()!;
              return ListView(
                shrinkWrap: true,
                physics: BouncingScrollPhysics(),
                padding: EdgeInsets.symmetric(horizontal: 20, vertical:
36),
                children: [
                  SizedBox(height: 16),
                  // Section 1 - Welcome Back
                  Container(
                    margin: EdgeInsets.only(bottom: 16),
                    width: MediaQuery.of(context).size.width,
                    child: Row(
                      mainAxisAlignment: MainAxisAlignment.spaceBetween,
                      children: [
                        Row(
                          children: [
                            ClipOval(
                              child: Container(
                                width: 42,
                                height: 42,
                                child: Image.network(
                                  (user["avatar"] == null ||
                                    user['avatar'] == "")
                                    ? "https://ui-
avatars.com/api/?name=${user['name']}"
                                    : user['avatar'],
                                  fit: BoxFit.cover,
                                ),
                              ),
                            SizedBox(width: 24),
                            Column(
                              crossAxisAlignment:
CrossAxisAlignment.start,
                              children: [
                                Text(
                                  "Selamat datang kembali",
                                  style: TextStyle(
                                    fontSize: 12,
                                    color: AppColor.secondarySoft,
                                  ),
                                ),

```

```

        ),
        SizedBox(height: 4),
        Text(
          user["name"],
          style: TextStyle(
            fontWeight: FontWeight.w500,
            fontFamily: 'poppins',
          ),
        ),
      ],
    ),
  ],
),
InkWell(
  onTap: () => {controller.logout()},
  child: Text("Logout"),
),
],
),
),
// section 2 - card
StreamBuilder<QuerySnapshot<Map<String, dynamic>>>(
  stream: controller.streamTodo().asBroadcastStream(),
  builder: (context, snapshot) {
    // #TODO: make skeleton
    switch (snapshot.connectionState) {
      case ConnectionState.waiting:
        return Center(child:
CircularProgressIndicator());
      case ConnectionState.active:
      case ConnectionState.done:
        var todoLengthData = snapshot.data?.docs.length;
        return Container(
          width: MediaQuery.of(context).size.width,
          padding: EdgeInsets.only(
            left: 24, top: 24, right: 24, bottom: 16),
          decoration: BoxDecoration(
            color: AppColor.primary,
            borderRadius: BorderRadius.circular(8),
            image: DecorationImage(
              image:
                AssetImage('assets/images/pattern-
1.png'),
              fit: BoxFit.cover,
            ),
          ),
          child: Column(
            crossAxisAlignment:
CrossAxisAlignment.start,
            children: [
              // job
              Text(
                user["email"],
                style: TextStyle(
                  color: Colors.white,
                  fontFamily: 'poppins',

```

```

fontWeight: FontWeight.w500,
    ),
  ),
  SizedBox(
    height: 20,
  ),
  Container(
    width:
MediaQuery.of(context).size.width,
    padding: EdgeInsets.symmetric(
      horizontal: 8,
      vertical: 16,
    ),
    decoration: BoxDecoration(
      color: AppColor.primarySoft,
      borderRadius:
BorderRadius.circular(8),
    ),
    child: Row(
      children: [
        Expanded(
          child: Column(
            children: [
              Container(
                margin:
                EdgeInsets.only(bottom:
6),
                child: Text(
                  "Jumlah Todo",
                  style: TextStyle(
                    fontSize: 12,
                    color: Colors.white,
                  ),
                ),
              ),
              Text(
                "${todoLengthData.toString()}
",
                style: TextStyle(
                  fontSize: 16,
                  fontWeight:
FontWeight.w700,
                  color: Colors.white,
                ),
              ),
            ],
          ),
        ),
      ],
    ),
  );
default:
  return SizedBox();

```

```

        }
      },
    ),
    SizedBox(
      height: 14,
    ),
    Container(
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Text(
            "List Todo Terbaru",
            style: TextStyle(
              fontFamily: "poppins",
              fontWeight: FontWeight.w600,
            ),
          ),
          TextButton(
            onPressed: () => Get.toNamed(Routes.ALL_TODO),
            child: Text("tampilkan semua"),
            style: TextButton.styleFrom(
              primary: AppColor.primary,
            ),
          ),
        ],
      ),
    ),
  ),
),

StreamBuilder<QuerySnapshot<Map<String, dynamic>>>>(
  stream:
controller.streamLastTodo().asBroadcastStream(),
  builder: (context, snapshot) {
    switch (snapshot.connectionState) {
      case ConnectionState.waiting:
        return Center(child:
CircularProgressIndicator());
      case ConnectionState.active:
      case ConnectionState.done:
        List<QueryDocumentSnapshot<Map<String,
dynamic>>>

          listResults = snapshot.data!.docs;
          return ListView.separated(
            itemCount: listResults.length,
            shrinkWrap: true,
            physics: BouncingScrollPhysics(),
            separatorBuilder: (context, index) =>
              SizedBox(height: 16),
            itemBuilder: (context, index) {
              var todoData = listResults[index].data();
              return InkWell(
                onTap: () => {
                  Get.toNamed(
                    Routes.DETAIL_TODO,
                    arguments: {
                      "id": "${todoData["task_id"]}",

```



```

        "title": "${todoData["title"]}",
        "description":
            "${todoData["description"]}",
    },
),
},
borderRadius: BorderRadius.circular(8),
child: Container(
    width:
MediaQuery.of(context).size.width,
    decoration: BoxDecoration(
        borderRadius:
BorderRadius.circular(8),
        border: Border.all(
            width: 1,
            color: AppColor.primaryExtraSoft,
        ),
    ),
padding: EdgeInsets.only(
    left: 24,
    top: 20,
    right: 29,
    bottom: 20),
child: Row(
    mainAxisAlignment:
        MainAxisAlignment.spaceBetween,
    children: [
        Column(
            crossAxisAlignment:
                CrossAxisAlignment.start,
            children: [
                Text(
                    (todoData["title"] == null)
                        ? "-"
                        :
"${todoData["title"]}",
                    style: TextStyle(fontSize:
12),
                ),
                Text(
                    "${todoData["created_at"]}",
                    style: TextStyle(fontSize:
12),
                ),
            ],
        ),
    ],
),
);
},
);
default:
    return SizedBox();
}
}),

```

```

        SizedBox(height: 64),
      ],
    );
    case ConnectionState.waiting:
      return Center(child: CircularProgressIndicator());
    default:
      return Center(child: Text("Error"));
  }
}),
floatingActionButton: FloatingActionButton(
  onPressed: () {
    Get.toNamed(Routes.ADD_TODO);
  },
  child: Icon(
    Icons.add,
  ),
),
);
}
}

```

- Selesai, silahkan di jalankan dengan menggunakan 'flutter run'.

9. *Create Data di Flutter*

- Buat halaman add_todo dengan menjalankan kode berikut ini di terminal.

```
get create page:add_todo
```

Jika sudah maka pada folder lib/app/modules akan ada folder baru dengan nama add_todo. Langkah selanjutnya adalah memberikan kode untuk file add_todo_controller.dart seperti berikut ini:

Deklarasi variabel yang dibutuhkan dalam view

```

RxBool isLoading = false.obs;
RxBool isLoadingCreateTodo = false.obs;

TextEditingController titleC = TextEditingController();
TextEditingController descriptionC = TextEditingController();

FirebaseAuth auth = FirebaseAuth.instance;
FirebaseFirestore firestore = FirebaseFirestore.instance;

```

Membuat fungsi dengan nama addTodo dan createTodo, pada fungsi addTodo digunakan untuk melakukan pengecekan terhadap input yang

diberikan oleh pengguna, sedangkan untuk createTodoData digunakan untuk menambah data pada Firestore Database.

```
Future<void> addTodo() async {
  if (titleC.text.isNotEmpty && descriptionC.text.isNotEmpty) {
    isLoading.value = true;

    if (isLoadingCreateTodo.isFalse) {
      await createTodoData();
      isLoading.value = false;
    }
  } else {
    isLoading.value = false;
    CustomToast.errorToast('Error', 'you need to fill all form');
  }
}

createTodoData() async {
  isLoadingCreateTodo.value = true;
  String adminEmail = auth.currentUser!.email!;
  try {
    String uid = auth.currentUser!.uid;
    CollectionReference<Map<String, dynamic>> childrenCollection =
      await
    firestore.collection("users").doc(uid).collection("todos");

    var uuidTodo = Uuid().v1();

    await childrenCollection.doc(uuidTodo).set({
      "task_id": uuidTodo,
      "title": titleC.text,
      "description": descriptionC.text,
      "created_at": DateTime.now().toIso8601String(),
    });

    Get.back(); //close dialog
    Get.back(); //close form screen
    CustomToast.successToast('Success', 'Berhasil menambahkan todo');

    isLoadingCreateTodo.value = false;
  } on FirebaseAuthException catch (e) {
    isLoadingCreateTodo.value = false;
    CustomToast.errorToast('Error', 'error : ${e.code}');
  } catch (e) {
    isLoadingCreateTodo.value = false;
    CustomToast.errorToast('Error', 'error : ${e.toString()}');
  }
}
```

Tambahkan fungsi onClose untuk menghapus Controller yang sudah dideklarasikan sebelumnya dengan menggunakan fungsi dispose().

```
@override
  onClose() {
    titleC.dispose();
    descriptionC.dispose();
  }
```

Berikut kode lengkap untuk add_todo_controller.dart

```
class AddTodoController extends GetxController {
  @override
  onClose() {
    titleC.dispose();
    descriptionC.dispose();
  }

  RxBool isLoading = false.obs;
  RxBool isLoadingCreateTodo = false.obs;

  TextEditingController titleC = TextEditingController();
  TextEditingController descriptionC = TextEditingController();

  FirebaseAuth auth = FirebaseAuth.instance;
  FirebaseFirestore firestore = FirebaseFirestore.instance;

  Future<void> addTodo() async {
    if (titleC.text.isNotEmpty && descriptionC.text.isNotEmpty) {
      isLoading.value = true;

      if (isLoadingCreateTodo.isFalse) {
        await createTodoData();
        isLoading.value = false;
      } else {
        isLoading.value = false;
        CustomToast.errorToast('Error', 'you need to fill all form');
      }
    }
  }

  createTodoData() async {
    isLoadingCreateTodo.value = true;
    String adminEmail = auth.currentUser!.email!;
    try {
      String uid = auth.currentUser!.uid;
      CollectionReference<Map<String, dynamic>> childrenCollection =
        await
        firestore.collection("users").doc(uid).collection("todos");

      var uuidTodo = Uuid().v1();

      await childrenCollection.doc(uuidTodo).set({
        "task_id": uuidTodo,
        "title": titleC.text,
```

```

        "description": descriptionC.text,
        "created_at": DateTime.now().toIso8601String(),
    });

    Get.back(); //close dialog
    Get.back(); //close form screen
    CustomToast.successToast('Success', 'Berhasil menambahkan todo');

    isLoadingCreateTodo.value = false;
  } on FirebaseAuthException catch (e) {
    isLoadingCreateTodo.value = false;
    CustomToast.errorToast('Error', 'error : ${e.code}');
  } catch (e) {
    isLoadingCreateTodo.value = false;
    CustomToast.errorToast('Error', 'error : ${e.toString()}');
  }
}
}

```

- Buat halaman `add_todo_view.dart` menjadi seperti berikut ini

```

class AddTodoView extends GetView<AddTodoController> {
  const AddTodoView({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(
          'Tambah Todo',
          style: TextStyle(
            color: AppColor.secondary,
            fontSize: 14,
          ),
        ),
        leading: IconButton(
          onPressed: () => Get.back(),
          icon: SvgPicture.asset('assets/icons/arrow-left.svg'),
        ),
        backgroundColor: Colors.white,
        elevation: 0,
        centerTitle: true,
        bottom: PreferredSize(
          preferredSize: Size.fromHeight(1),
          child: Container(
            width: MediaQuery.of(context).size.width,
            height: 1,
            color: AppColor.secondaryExtraSoft,
          ),
        ),
      ),
      body: ListView(
        shrinkWrap: true,

```

```

physics: BouncingScrollPhysics(),
padding: EdgeInsets.all(20),
children: [
  CustomInput(
    controller: controller.titleC,
    label: 'Nama Todo',
    hint: 'Menyelesaikan tugas',
  ),
  CustomInput(
    controller: controller.descriptionC,
    label: 'Deskripsi Todo',
    hint: 'Diselesaikan sebelum tanggal 25',
  ),
  SizedBox(height: 32),
  Container(
    width: MediaQuery.of(context).size.width,
    child: Obx(
      () => ElevatedButton(
        onPressed: () {
          if (controller.isLoading.isFalse) {
            controller.addToDo();
          }
        },
        child: Text(
          (controller.isLoading.isFalse) ? 'Tambah todo' :
'Loading...',
          style: TextStyle(
            fontSize: 16,
            fontFamily: 'poppins',
          ),
        ),
        style: ElevatedButton.styleFrom(
          primary: AppColor.primary,
          padding: EdgeInsets.symmetric(vertical: 18),
          elevation: 0,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(8),
          ),
        ),
      ),
    ),
  ),
),
],
),
);
}
}

```

- Selesai, silahkan di jalankan dengan menggunakan 'flutter run'.

10. *Read All* Data di Flutter

- Buat halaman `all_todo` dengan menjalankan kode berikut ini di terminal.

```
get create page:all_todo
```

Jika sudah maka pada folder `lib/app/modules` akan ada folder baru dengan nama `all_todo`. Langkah selanjutnya adalah memberikan kode untuk file `all_todo_controller.dart` seperti berikut ini:

Deklarasi variabel untuk `auth` dan `firebase`

```
FirebaseAuth auth = FirebaseAuth.instance;  
Firestore firestore = FirebaseFirestore.instance;
```

Selanjutnya membuat fungsi untuk memanggil data dari `firebase` dengan nama `getAllResult`, pada fungsi ini akan memanggil data `todo` yang ada pada pengguna yang sudah login.

```
Future<QuerySnapshot<Map<String, dynamic>>> getAllResult() async {  
  String uid = auth.currentUser!.uid;  
  QuerySnapshot<Map<String, dynamic>> query = await firestore  
    .collection("users")  
    .doc(uid)  
    .collection("todos")  
    .orderBy(  
      "created_at",  
      descending: true,  
    )  
    .get();  
  return query;  
}
```

Berikut kode lengkap untuk `add_todo_controller.dart`

```
class AllTodoController extends GetxController {  
  FirebaseAuth auth = FirebaseAuth.instance;  
  FirebaseFirestore firestore = FirebaseFirestore.instance;  
  
  Future<QuerySnapshot<Map<String, dynamic>>> getAllResult() async {  
    String uid = auth.currentUser!.uid;  
    QuerySnapshot<Map<String, dynamic>> query = await firestore  
      .collection("users")  
      .doc(uid)  
      .collection("todos")  
      .orderBy(  
        "created_at",  
        descending: true,  
      )  
      .get();  
    return query;  
  }
```

```

    )
    .get();
    return query;
  }
}

```

- Buat halaman `all_todo_view.dart` menjadi seperti berikut ini

```

import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/material.dart';
import 'package:flutter_crud_firestore/app/routes/app_pages.dart';
import 'package:flutter_crud_firestore/app/Utils/app_color.dart';
import 'package:flutter_svg/flutter_svg.dart';

import 'package:get/get.dart';

import '../controllers/all_todo_controller.dart';

class All_TODO_View extends GetView<All_TODO_Controller> {
  const All_TODO_View({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(
          'Semua Data TODO',
          style: TextStyle(
            color: AppColor.secondary,
            fontSize: 14,
          ),
        ),
        leading: IconButton(
          onPressed: () => Get.back(),
          icon: SvgPicture.asset('assets/icons/arrow-left.svg'),
        ),
        backgroundColor: Colors.white,
        elevation: 0,
        centerTitle: true,
        bottom: PreferredSize(
          preferredSize: Size.fromHeight(1),
          child: Container(
            width: MediaQuery.of(context).size.width,
            height: 1,
            color: AppColor.secondaryExtraSoft,
          ),
        ),
      ),
      body: GetBuilder<All_TODO_Controller>(
        builder: (con) {
          return FutureBuilder<QuerySnapshot<Map<String, dynamic>>>>(
            future: controller.getAllResult(),
            builder: (context, snapshot) {

```



```

        switch (snapshot.connectionState) {
            case ConnectionState.waiting:
                return Center(child: CircularProgressIndicator());
            case ConnectionState.active:
            case ConnectionState.done:
                var data = snapshot.data!.docs;
                return ListView.separated(
                    itemCount: data.length,
                    shrinkWrap: true,
                    physics: BouncingScrollPhysics(),
                    padding: EdgeInsets.all(20),
                    separatorBuilder: (context, index) =>
                        SizedBox(height: 16),
                    itemBuilder: (context, index) {
                        var todoData = data[index].data();
                        String desc;
                        return InkWell(
                            onTap: () => {
                                Get.toNamed(
                                    Routes.DETAIL_TODO,
                                    arguments: {
                                        "id": "${todoData["task_id"]}",
                                        "title": "${todoData["title"]}",
                                        "description":
                                            "${todoData["description"]}",
                                    },
                                )
                            },
                            borderRadius: BorderRadius.circular(8),
                            child: Container(
                                width: MediaQuery.of(context).size.width,
                                decoration: BoxDecoration(
                                    borderRadius: BorderRadius.circular(8),
                                    border: Border.all(
                                        width: 1,
                                        color: AppColor.primaryExtraSoft,
                                    ),
                                ),
                                padding: EdgeInsets.only(
                                    left: 24, top: 20, right: 29, bottom: 20),
                                child: Row(
                                    mainAxisAlignment:
                                        MainAxisAlignment.spaceBetween,
                                    children: [
                                        Column(
                                            crossAxisAlignment:
                                                CrossAxisAlignment.start,
                                            children: [
                                                Text(
                                                    (todoData["title"] == null)
                                                        ? "-"
                                                        : "${todoData["title"]}",
                                                    style: TextStyle(fontSize: 12),
                                                ),
                                                Text(
                                                    "${todoData["created_at"]}",

```

```

        style: TextStyle(fontSize: 12),
      ),
    ],
  ),
],
),
),
);
},
);
default:
  return SizedBox();
}
},
);
},
);
}
}

```

- Selesai, silahkan di jalankan dengan menggunakan 'flutter run'.

11. *Detail* Data di Flutter

- Buat halaman detail_todo dengan menjalankan kode berikut ini di terminal.

```
get create page:detail_todo
```

Jika sudah maka pada folder lib/app/modules akan ada folder baru dengan nama detail_todo. Langkah selanjutnya adalah memberikan kode untuk file detail_todo_controller.dart seperti berikut ini:

Deklarasi variabel untuk input dan juga form, serta mengambil argument / data yang dikirim dari halaman sebelumnya dengan menggunakan Get.Arguments.

```

final Map<String, dynamic> argsData = Get.arguments;

RxBool isLoading = false.obs;
RxBool isLoadingCreateTodo = false.obs;

TextEditingController titleC = TextEditingController();
TextEditingController descriptionC = TextEditingController();

FirebaseAuth auth = FirebaseAuth.instance;
FirebaseFirestore firestore = FirebaseFirestore.instance;

```

Buatkan fungsi untuk memberikan nilai pada variabel titleC dan descriptionC, sehingga bisa mengetahui nilai dari masing-masing detail todo.

```
@override
void onInit() {
  super.onInit();
  titleC.text = argsData["title"];
  descriptionC.text = argsData["description"];
}
```

Seperti pada halaman add_todo, kita juga harus menambahkan fungsi onClose yang digunakan untuk menghilangkan Controller yang sudah kita deklarasikan sebelumnya.

```
@override
onClose() {
  titleC.dispose();
  descriptionC.dispose();
}
```

Berikut kode lengkap untuk add_todo_controller.dart

```
class DetailTodoController extends GetxController {
  final Map<String, dynamic> argsData = Get.arguments;

  RxBool isLoading = false.obs;
  RxBool isLoadingCreateTodo = false.obs;

  TextEditingController titleC = TextEditingController();
  TextEditingController descriptionC = TextEditingController();

  FirebaseAuth auth = FirebaseAuth.instance;
  FirebaseFirestore firestore = FirebaseFirestore.instance;

  @override
  onClose() {
    titleC.dispose();
    descriptionC.dispose();
  }

  @override
  void onInit() {
    super.onInit();
    titleC.text = argsData["title"];
    descriptionC.text = argsData["description"];
  }
}
```

- Buat halaman detail_todo_view.dart menjadi seperti berikut ini

```
class Detail_TODOView extends GetView<Detail_TODOController> {
  const Detail_TODOView({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(
          'Detail TODO',
          style: TextStyle(
            color: AppColor.secondary,
            fontSize: 14,
          ),
        ),
      ),
      leading: IconButton(
        onPressed: () => Get.back(),
        icon: SvgPicture.asset('assets/icons/arrow-left.svg'),
      ),
      actions: [
        TextButton(
          onPressed: () {
            Get.toNamed(Routes.EDIT_TODO, arguments:
controller.argsData);
          },
          child: Text('Edit'),
          style: TextButton.styleFrom(
            primary: AppColor.primary,
          ),
        ),
      ],
      backgroundColor: Colors.white,
      elevation: 0,
      centerTitle: true,
      bottom: PreferredSize(
        preferredSize: Size.fromHeight(1),
        child: Container(
          width: MediaQuery.of(context).size.width,
          height: 1,
          color: AppColor.secondaryExtraSoft,
        ),
      ),
    ),
    body: ListView(
      shrinkWrap: true,
      physics: BouncingScrollPhysics(),
      padding: EdgeInsets.all(20),
      children: [
        CustomInput(
          controller: controller.titleC,
          label: 'Nama TODO',
          hint: 'Menyelesaikan tugas',
          disabled: true,
```

```

    ),
    CustomInput(
      controller: controller.descriptionC,
      label: 'Deskripsi Todo',
      hint: 'Diselesaikan sebelum tanggal 25',
      disabled: true,
    ),
  ],
),
);
}
}

```

- Selesai, silahkan di jalankan dengan menggunakan 'flutter run'.

12. *Edit* Data di Flutter

- Buat halaman edit_todo dengan menjalankan kode berikut ini di terminal.

```
get create page:edit_todo
```

Jika sudah maka pada folder lib/app/modules akan ada folder baru dengan nama edit_todo. Langkah selanjutnya adalah memberikan kode untuk file edit_todo_controller.dart seperti berikut ini:

Deklarasi variabel untuk input, argument dan juga firestore

```

final Map<String, dynamic> argsData = Get.arguments;

RxBool isLoading = false.obs;
RxBool isLoadingCreateTodo = false.obs;

TextEditingController titleC = TextEditingController();
TextEditingController descriptionC = TextEditingController();

FirebaseAuth auth = FirebaseAuth.instance;
FirebaseFirestore firestore = FirebaseFirestore.instance;

```

Buatkan fungsi untuk memberikan nilai pada variabel titleC dan descriptionC, sehingga bisa mengetahui nilai dari masing-masing detail todo.

```

@override
void onInit() {
  super.onInit();
}

```

```
titleC.text = argsData["title"];
descriptionC.text = argsData["description"];
}
```

Seperti pada halaman `add_todo`, kita juga harus menambahkan fungsi `onClose` yang digunakan untuk menghilangkan Controller yang sudah kita deklarasikan sebelumnya.

```
@override
onClose() {
  titleC.dispose();
  descriptionC.dispose();
}
```

Membuat fungsi dengan nama `editTodo` dan `editTodoData`, pada fungsi `editTodo` digunakan untuk melakukan pengecekan terhadap input yang diberikan oleh pengguna, sedangkan untuk `editTodoData` digunakan untuk memperbarui data pada Firestore Database.

```
Future<void> editTodo() async {
  if (titleC.text.isNotEmpty && descriptionC.text.isNotEmpty) {
    isLoading.value = true;

    if (isLoadingCreateTodo.isFalse) {
      await editTodoData();
      isLoading.value = false;
    }
  } else {
    isLoading.value = false;
    CustomToast.errorToast('Error', 'you need to fill all form');
  }
}

editTodoData() async {
  isLoadingCreateTodo.value = true;
  String adminEmail = auth.currentUser!.email!;
  try {
    String uid = auth.currentUser!.uid;
    CollectionReference<Map<String, dynamic>> childrenCollection =
      await
    firestore.collection("users").doc(uid).collection("todos");

    await childrenCollection.doc(argsData["id"]).update({
      "title": titleC.text,
      "description": descriptionC.text,
    });

    Get.back(); //close dialog
  } catch (e) {
    CustomToast.errorToast('Error', e.toString());
  }
}
```

```

        Get.back(); //close form screen
        CustomToast.successToast('Success', 'Berhasil memperbarui todo');

        isLoadingCreateTodo.value = false;
    } on FirebaseAuthException catch (e) {
        isLoadingCreateTodo.value = false;
        CustomToast.errorToast('Error', 'error : ${e.code}');
    } catch (e) {
        isLoadingCreateTodo.value = false;
        CustomToast.errorToast('Error', 'error : ${e.toString()}');
    }
}

```

Berikut kode lengkap untuk edit_todo_controller.dart

```

class EditTodoController extends GetxController {
    final Map<String, dynamic> argsData = Get.arguments;

    RxBool isLoading = false.obs;
    RxBool isLoadingCreateTodo = false.obs;

    TextEditingController titleC = TextEditingController();
    TextEditingController descriptionC = TextEditingController();

    FirebaseAuth auth = FirebaseAuth.instance;
    FirebaseFirestore firestore = FirebaseFirestore.instance;

    @override
    onClose() {
        titleC.dispose();
        descriptionC.dispose();
    }

    @override
    void onInit() {
        super.onInit();
        titleC.text = argsData["title"];
        descriptionC.text = argsData["description"];
    }

    Future<void> editTodo() async {
        if (titleC.text.isNotEmpty && descriptionC.text.isNotEmpty) {
            isLoading.value = true;

            if (isLoadingCreateTodo.isFalse) {
                await editTodoData();
                isLoading.value = false;
            }
        } else {
            isLoading.value = false;
            CustomToast.errorToast('Error', 'you need to fill all form');
        }
    }
}

```

```

    }

    editTodoData() async {
      isLoadingCreateTodo.value = true;
      String adminEmail = auth.currentUser!.email!;
      try {
        String uid = auth.currentUser!.uid;
        CollectionReference<Map<String, dynamic>> childrenCollection =
          await
            firestore.collection("users").doc(uid).collection("todos");

        await childrenCollection.doc(argsData["id"]).update({
          "title": titleC.text,
          "description": descriptionC.text,
        });

        Get.back(); //close dialog
        Get.back(); //close form screen
        CustomToast.successToast('Success', 'Berhasil memperbarui todo');

        isLoadingCreateTodo.value = false;
      } on FirebaseAuthException catch (e) {
        isLoadingCreateTodo.value = false;
        CustomToast.errorToast('Error', 'error : ${e.code}');
      } catch (e) {
        isLoadingCreateTodo.value = false;
        CustomToast.errorToast('Error', 'error : ${e.toString()}');
      }
    }
  }
}

```

- Buat halaman edit_data_view.dart menjadi seperti berikut ini

```

class EditTodoView extends GetView<EditTodoController> {
  const EditTodoView({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(
          'Edit Todo',
          style: TextStyle(
            color: AppColor.secondary,
            fontSize: 14,
          ),
        ),
      ),
      leading: IconButton(
        onPressed: () => Get.back(),
        icon: SvgPicture.asset('assets/icons/arrow-left.svg'),
      ),
      backgroundColor: Colors.white,
      elevation: 0,
    );
  }
}

```



```

        centerTitle: true,
        bottom: PreferredSize(
          preferredSize: Size.fromHeight(1),
          child: Container(
            width: MediaQuery.of(context).size.width,
            height: 1,
            color: AppColor.secondaryExtraSoft,
          ),
        ),
      ),
    ),
    body: ListView(
      shrinkWrap: true,
      physics: BouncingScrollPhysics(),
      padding: EdgeInsets.all(20),
      children: [
        CustomInput(
          controller: controller.titleC,
          label: 'Nama Todo',
          hint: 'Menyelesaikan tugas',
        ),
        CustomInput(
          controller: controller.descriptionC,
          label: 'Deskripsi Todo',
          hint: 'Diselesaikan sebelum tanggal 25',
        ),
        SizedBox(height: 32),
        Container(
          width: MediaQuery.of(context).size.width,
          child: Obx(
            () => ElevatedButton(
              onPressed: () {
                if (controller.isLoading.isFalse) {
                  controller.editTodo();
                }
              },
              child: Text(
                (controller.isLoading.isFalse) ? 'Edit todo' :
'Loading...',
                style: TextStyle(
                  fontSize: 16,
                  fontFamily: 'poppins',
                ),
              ),
              style: ElevatedButton.styleFrom(
                primary: AppColor.primary,
                padding: EdgeInsets.symmetric(vertical: 18),
                elevation: 0,
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(8),
                ),
              ),
            ),
          ),
        ],
      ),
    ),
  ),
)

```

```
);
}
}
```

- Selesai, silahkan di jalankan dengan menggunakan 'flutter run'.

13. Delete Data di Flutter

- Pada halaman detail kita akan menambahkan button yang digunakan untuk menghapus data dari todo. Tambahkan kode berikut ini pada detail_todo_controller.dart.

```
Future<void> deleteTodo() async {
  CustomAlertDialog.showPresenceAlert(
    title: "Hapus data todo",
    message: "Apakah anda ingin menghapus data todo ini ?",
    onCancel: () => Get.back(),
    onConfirm: () async {
      Get.back(); // close modal
      Get.back(); // back page

      try {
        String uid = auth.currentUser!.uid;
        await firestore
          .collection('users')
          .doc(uid)
          .collection('todos')
          .doc(argsData['id'])
          .delete();
        CustomToast.successToast('Success', 'Data todo berhasil
dihapus');
      } catch (e) {
        CustomToast.errorToast(
          "Error", "Error dikarenakan : ${e.toString()}");
      }
    },
  );
}
```

- Selanjutnya tambahkan file berikut pada bagian bawahnya 'CustomInput deskripsi' di detail_todo_view.dart.

```
ElevatedButton(
  onPressed: () {
    controller.deleteTodo();
  },
  child: Text(
    'Delete todo',
```

```

        style: TextStyle(
          fontSize: 16,
          fontFamily: 'poppins',
        ),
      ),
      style: ElevatedButton.styleFrom(
        primary: AppColor.warning,
        padding: EdgeInsets.symmetric(vertical: 18),
        elevation: 0,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(8),
        ),
      ),
    ),
  ),
),

```

- Selesai, silahkan di jalankan dengan menggunakan 'flutter run'.

14. Berikut beberapa tampilan dari aplikasi jika berhasil menjalankannya.

